

A coarsening method for bipartite networks via weight-constrained label propagation[☆]



Alan Valejo^{a,*}, Thiago Faleiros^b, Maria Cristina Ferreira de Oliveira^a,
Alneu de Andrade Lopes^a

^a Institute of Mathematical and Computer Sciences (ICMC), University of São Paulo (USP), P.O. Box 668, P.C. 14560-970, São Carlos, SP, Brazil

^b Department of Computer Science (CiC), University of Brasília (UnB), P.O. Box 4466, P.C. 70910-900, Brasília, DF, Brazil

ARTICLE INFO

Article history:

Received 21 May 2019

Received in revised form 14 February 2020

Accepted 18 February 2020

Available online 25 February 2020

Keywords:

Complex networks
Multilevel method
Large-scale networks
Bipartite networks
Community detection
Network coarsening
Network visualization

ABSTRACT

A multilevel method is a scalable strategy to solve optimization problems in large bipartite networks, which operates in three stages. Initially the input network is iteratively coarsened into a hierarchy of gradually smaller networks. Coarsening implies in collapsing vertices into so-called super-vertices which inherit properties of their originating vertices. An initial solution is obtained executing the target algorithm in the coarsest network. Finally, this solution is successively projected back over the inverse sequence of coarsened networks, up to the initial one, yielding an approximate final solution. Despite its potential applicability, the strategy faces several theoretical and practical limitations. Coarsening is usually attained following a user-defined policy to match vertices pairwise. However, the network reduction process is extremely slow and may yield degraded solutions due to propagation of poor matches. Additionally, proper parameterization of coarsening algorithms is difficult, as well as ensuring the super-vertices preserve the relevant properties. We address these issues with a near-linear complexity coarsening strategy based on weight-constrained label propagation. Our strategy collapses groups of vertices, rather than pairs, yielding faster and more extensive network reduction. Moreover, users may specify the desired size of the coarsest network and control super-vertex weights. The applicability of our solution is illustrated in multiple scenarios, namely: multilevel implementation of an existing high-cost community detection algorithm; as a direct community detection algorithm; finally, network visualization, in connection with force-directed graph drawing algorithms. Results provide empirical evidence on the potential of our proposal to foster novel applications of the multilevel method in bipartite networks.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

Bipartite, or two-layer, networks comprise a particular category of networks in which the set of vertices is split into two disjoint sub-sets (the layers) and edges can connect only vertices of different layers. This type of network often results from modeling pairwise relationships between two categories of entities, e.g. documents and terms [1], patient and gene expression (or clinical variables) [2], individuals and songs (or books, or films) [3], scientific papers and their authors [4,5]. Over the

last years, there has been a growing scientific interest in bipartite networks given their occurrence in many data analytics problems, such as community detection, text classification, and dimensionality reduction [6,7]. Their potential emerges from the capability of representing conceptual edges between different types of entities and their derived latent features, which supports a new perspective in modeling complex systems. Solving these and other problems requires finding a minimum (or maximum) cost in a solution space wherein the number of possible states is finite and usually exponential. Indeed, many theoretical problems defined in bipartite networks have been proven to be $\mathcal{NP}$ -complete or $\mathcal{NP}$ -hard [8].

The multilevel method has been investigated as a scalable strategy to solve partitioning and other hard problems in bipartite networks. Its underlying strategy is to execute the target algorithm on a reduced version of the network in order to obtain an approximate initial solution, which is then refined to yield the desired solution [6,9–12]. The method is executed in three phases, namely *coarsening*, *solution finding* with the *target algorithm* and

[☆] No author associated with this paper has disclosed any potential or pertinent conflicts which may be perceived to have impending conflict with this work. For full disclosure statements refer to <https://doi.org/10.1016/j.knosys.2020.105678>.

* Corresponding author.

E-mail addresses: alanvalejo@gmail.com (A. Valejo), thiagodepaulo@unb.br (T. Faleiros), cristina@icmc.usp.br (M.C.F. Oliveira), alneu@icmc.usp.br (A.A. Lopes).

uncoarsening, as illustrated in Fig. 1 [9,10]. Initially the input network is iteratively coarsened into a hierarchy of gradually smaller networks. The coarsening phase is at the core of the multilevel method, and takes two input parameters:

- a number of coarsening levels**, which defines the height of the hierarchy of coarsened networks, or how many coarsening iterations will be performed;
- a reduction factor**, which specifies the rate at which a network must be reduced when coarsened from one level (iteration) to the next one in the hierarchy.

Each coarsened network is a simplified version of the previous one in the hierarchy. Network reduction is usually attained by matching pairs of vertices into a single entity, referred to as super-vertices, according to a defined criterion. In the second phase, the target algorithm (or a target task) is executed in the coarsest network of the multilevel hierarchy to compute an initial solution to the problem. Finally, in a third and final phase this initial solution is successively projected back over the inverse hierarchy of coarsened networks, until a solution relative to the original network is obtained.

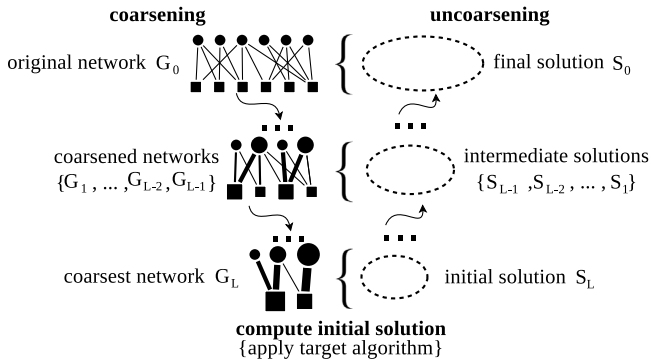


Fig. 1. Phases of a multilevel method: coarsening, initial solution, and uncoarsening. The number of coarsening levels (iterations) and network the reduction factor are input parameters.

Despite its potential, application of the multilevel method in bipartite networks faces theoretical and practical limitations. One such limitation is that network coarsening by collapsing vertex pairs often yields to degradation of relevant topological structures, compromising solution quality. Pairwise vertex collapsing also implies in a network being reduced by up to half its original size in a coarsening iteration, rendering reduction of very large networks extremely slow. As an additional limitation, existing coarsening solutions do not admit to enforce that super-vertices are size-constrained, or balanced regarding some user-defined property. Finally, setting the coarsening parameters *a priori* can be rather difficult. From an end-user point-of-view it would be more intuitive to inform the desired size, i.e. the number of vertices, of the coarsest network.

Here we address these issues in order to enhance the applicability of the multilevel method and foster the development of novel multilevel solutions for problems defined in bipartite networks. This paper brings a two-fold contribution, from both a theoretical and an applied perspectives:

1. We introduce a near-linear time coarsening strategy based on weight-constrained label propagation. Compared with existing solutions its major distinguishing features are: (i) it allows collapsing groups of vertices, rather than vertex pairs; (ii) it admits higher network reduction rates (on

the order of 60%–90%); and (iii) the aimed network reduction may be specified either by informing the number of coarsening levels and corresponding reduction factor or, alternatively, by informing the size (number of vertices) of the coarsest network.

2. We illustrate multiple applications of the proposed coarsening strategy. It has been used in a multilevel implementation of the high-cost bipartite Stochastic Block Model (biSBM) [13] community detection algorithm. An empirical investigation conducted on a representative set of synthetic and real-world networks found a considerable speed up with no significant loss in solution quality, as compared to established coarsening algorithms. We also show the strategy is easily adaptable for community detection in bipartite networks, yielding partitions that are statistically equivalent, regarding accuracy, to those obtained with well-known algorithms. Finally, we illustrate the potential applicability of the coarsening strategy for network visualization, discussing its usage in connection with network drawing algorithms to obtain informative layouts of large bipartite networks.

The remainder of the paper is organized as follows: in Section 2 we introduce the basic concepts and notation, as well as a description of the multilevel approach applied in bipartite networks. In Section 3 we review related work, with a critical discussion of current limitations of vertex matching strategies for coarsening. In Section 4 we introduce the proposed coarsening strategy and its implementation. Empirical results are reported in Section 6, covering a benchmark analysis of our proposal; an empirical study of its application in community detection, and an investigation of its potential to support visualization of large networks. Finally, in Section 7 we summarize our findings and discuss future work.

2. Background

2.1. Basic definitions

A network $G = (V, E, \sigma, \omega)$ is bipartite (or two-layer) if its set of vertices V is partitioned into two sub sets V^1 and V^2 , such that $V^1 \cap V^2 = \emptyset$ and $E \subseteq V^1 \times V^2$, where each V^j , $j \in \{1, 2\}$ is called a layer. Bipartite networks are also employed in modeling quadratic and nonsymmetric/unsymmetric matrices, implying in $|V^1| = |V^2|$ [14,15]. Let $n = |V|$ and $m = |E|$ denote, respectively, the number of vertices and the number of edges. An edge (u, v) may have an associated weight, denoted as $\omega(u, v)$ with $\omega : V \times V \rightarrow \mathbb{R}^*$; a vertex u may have an associated weight denoted as $\sigma(u)$ with $\sigma : V \rightarrow \mathbb{R}^*$.

The degree of a vertex $u \in V$, denoted κ_u , is defined as the total weight of its adjacent edges, i.e. $\kappa_u = \sum_{v \in V} \omega(u, v)$. The h -hop neighborhood of u , denoted $\Gamma_h(u)$, is formally defined as the vertices in set $\Gamma_h(u) = \{v \mid \text{there is a path of length } h \text{ between } u \text{ and } v\}$. Thus, the 1-hop neighborhood of u , $\Gamma_1(u)$, is the set of vertices adjacent to u ; the 2-hop neighborhood, $\Gamma_2(u)$, is the set of vertices 2-hops away from u , and so forth.

A similarity score $S(u, v)$ can be computed from a pair of vertices u and v , yielding values in the range $[0, 1] \subset \mathbb{R}$, from lowest (0) to highest (1) similarity. Two possible similarity scores are defined in Eqs. (1) and (2), known as common neighbors and weighted common neighbors [16], respectively.

$$S_{cn}(u, v) = |\Lambda(u, v)|, \Lambda(u, v) = \{\Gamma_1(u) \cap \Gamma_1(v)\} \quad (1)$$

$$S_{wcn}(u, v) = \sum_{z \in \Lambda(u, v)} \frac{\omega(u, z) + \omega(v, z)}{\log(1 + s(z))} \quad (2)$$

$$s(z) = \sum_{u \in \Gamma_1(z)} \omega(z, u)$$

2.2. Multilevel method in bipartite networks

Multilevel method allows executing complex algorithms on large-scale networks by reducing the size of the network on which the algorithm is to be applied. Let us consider a problem defined in a bipartite network $G_0(V_0, E_0, \omega_0)$, on which executing a target algorithm is prohibitive.

Given a desired number of coarsening levels L , the coarsening phase constructs a hierarchy of coarsened networks G_l from G_0 , $l \in [1, \dots, L-1]$, yielding intermediate approximations of the original network at multiple levels-of-detail. Coarsening requires two algorithms, namely *matching*, that defines which vertices will be collapsed, and *contraction*, which builds the reduced representation once the matching has been defined. A matching $M = \{V_k\}_{k=0}^K$ is a partition of V into K non-empty and disjoint subsets $V_i \in V$ such that $V = \bigcup V_i$, $1 \leq |V_i|$, $\forall i$ and $i \neq j \Rightarrow V_i \cap V_j = \emptyset$. Restrictions specify how vertices can be matched, e.g. a restriction such as $|V_i| = 2$, $\forall i$ implies that vertices must be paired. A vertex $u \in V_i$ is said to be *matched*, otherwise, if $\nexists V_i \in M | u \in V_i$ the vertex is said to be *unmatched*, or a *singleton*. An additional input parameter $\rho \in [0, 1] \subset \mathbb{R}$ determines the (maximum) network reduction factor. This parameter defines the cardinality of the matching M , denoted as ζ , where $|M| = |\cup V_i| = \zeta = \sum |V_i|$.

If, for instance, vertices are to be matched pairwise, then $\zeta = \lceil |V|/\rho \rceil$, ($\rho \in (0, 0.5]$). Considering, as an illustration, a network with $|V| = 100$ vertices and $\rho = 0.3$, then $\sum |V_i| = 60$ vertices (or 30 vertex pairs) can will be selected for matching. If $\rho = 0.5$, at each iteration the vertex set will be reduced by (at most) a factor of two, yielding a logarithmic decrease on the size of the networks along the multiple coarsening iterations.

The contraction algorithm constructs a coarser network by collapsing a set of matched vertices $\{u_1, \dots, u_n\}$ into a single *super-vertex* sV_i . Vertices $\{u_1, \dots, u_n\} \in V_l$ that compose a super-vertex $sV_i \in V_{l+1}$ are called predecessor-vertices of sV_i . A successor-network G_{l+1} will inherit the non-matched vertices from its predecessor-network G_l . In order for G_{l+1} to be a good proxy to its predecessor-network, the weight $\sigma(sV_i)$ of a super-vertex $sV_i = \{u_1, \dots, u_n\} \in V_{l+1}$ is computed as the sum of weights of its predecessor-vertices. Furthermore, any edges incident to vertices $\{u_1, \dots, u_n\} \in V_l$ are collapsed to obtain the so-called *super-edges* incident to sV_i .

The solution finding phase applies the target algorithm to compute a solution in the coarsest bipartite network G_L . Let S be the set of all possible solutions in the coarsest problem instance. Given an objective function $f : S \rightarrow \mathbb{R}$ (or $\mathbb{N}$) that assigns a cost to each solution in S , the aim is to find a state $s \in S$ with optimum cost. For instance, in the traveling salesman problem $f(s)$ expresses the length of a tour s , whereas in a community detection problem it denotes some measure of community quality.

Finally, the uncoarsening phase, also known as solution projection, successively transfers the solution available at a current level to the next level in the hierarchy. The solution obtained in the coarsest network G_L is successively projected through intermediate bipartite networks $G_{L-1}, G_{L-2}, \dots, G_1$ up to the initial network G_0 . A refinement heuristics can be applied to avoid local minima and improve solution quality.

3. Related work

A coarsened network must preserve the major topological features of its predecessor-network at any level, i.e., connections and weights of vertices and edges must reflect the corresponding global patterns of their predecessors. However, the process of matching vertices involves many choices and selecting a matching strategy that preserves the structural properties relevant to the problem at hand is a critical component of an effective

multilevel implementation. An inadequate strategy may degrade solution quality and increase its computational cost.

Several matching algorithms have been introduced for unipartite networks. Two popular ones, which visit vertices in random order and match vertices pairwise, are Random Matching (RM) and Heavy-Edge Matching (HEM) [17]. In RM, once an unmatched vertex u is visited, one of its unmatched neighbors v , if any, will be arbitrarily selected as the match. In HEM, a random vertex u is matched with the adjacent vertex v so that edge (u, v) has maximum weight over all adjacent edges to u .

These algorithms are not directly applicable to bipartite networks, as they do not distinguish between vertex types and therefore cannot enforce a matching that preserves the bipartite property that adjacent vertices are in different layers. However, a bipartite network $G = \{V^1 \cup V^2, E\}$ can be decomposed into two unipartite networks G^1 and G^2 , one for each layer, so that adjacent vertices in the bipartite network are made adjacent in their corresponding projection [18,19]. This enables executing algorithms such as RM and HEM in each one-mode projection, and the matchings thus obtained can be mapped back to the original bipartite network. Valejo et al. [6,11] rely on this strategy in a matching algorithm for bipartite networks called OPM, illustrated in Fig. 2.

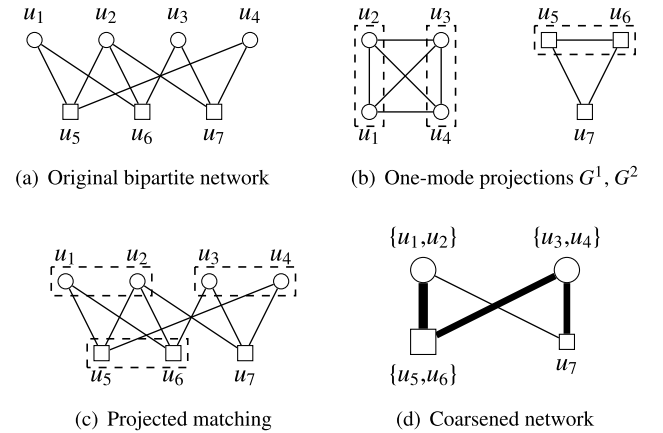


Fig. 2. OPM matching of a bipartite network resulting from matching its two one-mode projections with the RM matching algorithm.

Fig. 2(a) shows the input bipartite network and the corresponding one-mode projections of each layer are in Fig. 2(b). A possible outcome from applying the RM algorithm to G_1 and G_2 is the matching $M = \{\{u_1, u_2\}, \{u_3, u_4\}, \{u_5, u_6\}\}$, also indicated in Fig. 2(b). M is then transferred to the original bipartite network, Fig. 2(c). Finally, the contraction algorithm creates the coarsened bipartite network, shown in Fig. 2(d).

Valejo et al. [12] later introduced two novel algorithms designed specifically for matching vertices in bipartite networks, called *RGmb* and *Gmb*. They consider a local 2-hop neighborhood restriction which imposes that adjacent vertices cannot be matched and vertices can only be matched with others in their 2-hop neighborhood set. This restriction reduces the local search space and the computational cost of finding a match. The *RGmb* matching is illustrated in Fig. 3.

In an initial iteration, depicted in Fig. 3(a), a selected vertex u_1 is allowed to match non-adjacent vertices u_2 or u_4 , which belong to the 2-hop neighborhood of u_1 . Let us assume the pair $\{u_1, u_2\}$ is matched. In the next iteration, illustrated in Fig. 3(b), a selected vertex u_3 could match vertices u_2 or u_4 . Since u_2 has already been matched, the remaining choice is to match pair u_3 and u_4 . In the final iteration, the only choice left for the selected vertex u_6 is u_7 ,

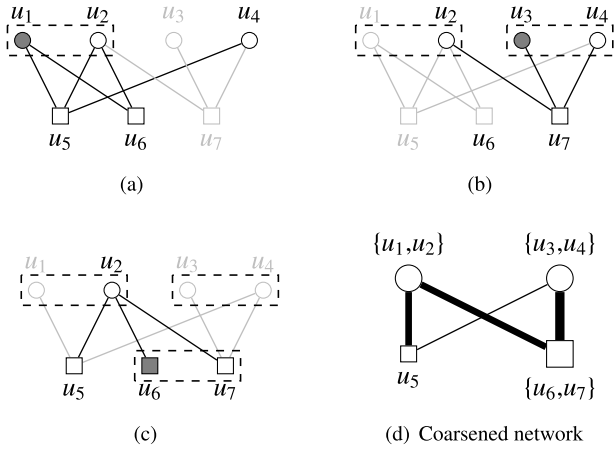


Fig. 3. RGMB matching for bipartite networks based on 2-hop neighborhoods.

Fig. 3(c). Finally, the contracting algorithm creates the coarsened bipartite network, Fig. 3(d).

Albeit algorithms OPM, RGMB, and GMB have been proved useful in several scenarios [6,11,12], strategies based on collapsing vertex pairs are inherently limited. For a start, there is an upper limit of 50% ($\rho = 0.5$) in the rate of network reduction at each iteration. Therefore, coarsening large networks with a high reduction factor demands many coarsening iterations, incurring in high processing and memory costs. Furthermore, vertices considered for matching at early stages are more likely to find good candidate matches that satisfy the similarity function than those considered later in the process. As a result, matching quality tends to degrade towards the final iterations, possibly introducing inconsistencies that will be propagated to upper coarsening levels. In networks with community structures, for instance, vertices from distinct communities may be matched, fragmenting the original topological structure. k -cliques are another relevant topological pattern that may be gradually degraded by low quality matches.

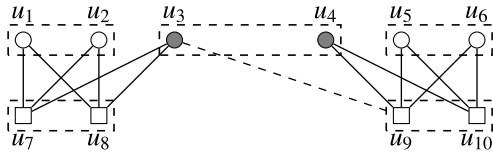


Fig. 4. Matching pair $\{u_3, u_4\}$ is a poor choice, as the resulting super-vertex will have predecessor vertices from distinct communities or cliques.

The problem is illustrated in Fig. 4 on a network with two communities and two bicliques. Whereas suitable matching choices have been made in earlier iterations, e.g. pairs $\{u_1, u_2\}$ or $\{u_5, u_6\}$, in the final iteration the single choice for the algorithm is to match pair $\{u_3, u_4\}$. This choice will, at the next coarsening level, degrade the biclique and community structures, since it yields a super-vertex that includes vertices from distinct communities and bicliques.

An additional issue is that super-vertices obtained with pairwise matching tend to be highly unbalanced. At each coarsening level, heavier super-vertices are likely to collapse with their lighter neighboring vertices and super-vertices, so they become even heavier and given higher matching priority in subsequent coarsening levels. Fig. 5 illustrates the problem, in which vertex pair $\{u_1, u_2\}$ is given priority over pair $\{u_2, u_3\}$, even though the number of common neighbors is the same for both pairs. In some applications it may be useful to constrain the size of the super-vertices, or to ensure they are balanced regarding some property.

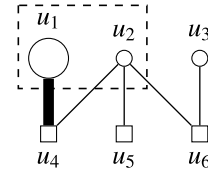


Fig. 5. The priority choice $\{u_1, u_2\}$ over the alternative $\{u_1, u_3\}$ will produce a highly unbalanced super-vertex.

For instance, in solving a document clustering problem it may be desirable to constrain super-vertex (cluster) size. Nonetheless, existing matching algorithms do not support such requirements.

Furthermore, existing matching algorithms require the number of coarsening levels as an input parameter. However, how many levels are necessary to attain a desired network reduction factor is typically unknown *a priori*. From a user perspective, it would be more intuitive to specify the desired size of the coarsest network. In a community detection problem, for instance, this would coincide with the target number of communities; in a visualization problem it would be natural to specify a final network size that fits the screen space, rather than informing how many coarsening levels and which reduction factor.

4. Coarsening via weight-constraint and label propagation

This section introduces a coarsening strategy (split into matching and contraction algorithms) for bipartite networks that relies on weight-constrained label propagation. The proposed matching and contraction algorithms are introduced and then employed to implement two distinct coarsening variations. Two premises establish the major distinction between current state-of-the-art coarsening algorithms and the ones proposed here:

1. Matchings are not restricted to vertices yet unmatched, i.e. both matched and unmatched vertices are allowed to form a new match.
2. The weights of super-vertices obtained from the matchings can be constrained.

The first premise allows super-vertices to collapse more than two vertices at once, so that vertices may be grouped which have common properties or play similar roles in the network, with multiple advantages. Greater network reduction factors may be attained at each coarsening level, e.g., up to 60%–90% depending on the weight constraints imposed on super-vertices. Therefore, fewer coarsening levels are necessary to attain a desired reduction rate, decreasing memory requirements and execution times. Finally, vertices considered for matching later in the process will not be restricted to a few choices. Overall, this approach yields a more consistent and compact coarsening hierarchy. The second restriction allows imposing an upper limit on the super-vertices weights, which is useful to generate super-vertices compatible with the original network structures and to prevent super-vertex weights deviating significantly from the average.

As matching algorithms must be fast, a cost effective approach is required to handle the weight constraints. In the proposed algorithm, Eq. (3) defines the maximum weight of a super-vertex in layer j and coarsening level l (henceforth the subscripted l is suppressed for simplification):

$$S_{max}^j = \lceil (1 + \mu) \frac{|V_0^j|}{\zeta} \rceil, \quad (3)$$

$$\zeta = \lceil |V^j| (1 - \rho) \rceil \quad (4)$$

where parameter $\mu \in (0, \zeta] \subset \mathbb{R}$ places an upper bound in the weight constraint, where $|V_0^j|$ is the number of vertices in each layer $j \in \{1, 2\}$ at the initial coarsening level $l = 0$. From Eq. (3), $\mu = \zeta$ implies $S_{max} \approx |V_0^j|$, i.e., a super-vertex $sV \in V_{l+1}$ may include all vertices $\in V_l$. A vertex v with weight $\sigma(v)$ can be matched with a vertex $u \in sV_i$ if Eq. (5) is satisfied:

$$\sigma(sV_i) + \sigma(v) \leq S_{max}^j, \quad (5)$$

A super-vertex sV_i is said to be *overloaded* if $\sigma(sV_i) > S_{max}^j$ and *underloaded* if $\sigma(sV_i) \leq S_{max}^j$. Notice that the value S_{max}^j is updated at each coarsening level and depends on ζ , which makes this formulation problem-specific. Fig. 6 illustrates how the upper bound parameter μ^j impacts the super-vertex weights S_{max}^j considering one layer ($j = 1$) of a bipartite network with $|V^1| = 100,000$ and reduction factor $\rho^1 = 0.5$.

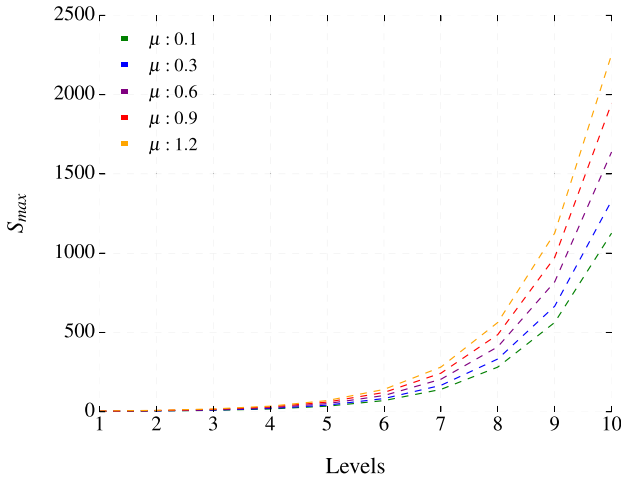


Fig. 6. Impact of the upper bound parameter μ on the constrained weights of super-vertices S_{max}^j along the coarsening levels.

The impact is limited along the initial coarsening levels; in contrast, high values of μ lead to heavy and probably unbalanced super-vertices in the final levels. In the particular case of $\mu = 1.0$ and $\rho = 0.5$ one observes a similar behavior to standard pairwise matching algorithms *OPM*, *GMB* and *RGMb*. For instance, in the first level $\zeta^1 = \lceil 100,000 \cdot 0.5 \rceil$, then $S_{max}^1 = \lceil 1 \cdot 100,000 / 50,000 \rceil = 2$, i.e., each super-vertex is formed by a pair of vertices, or $|V_i| = 2 \forall V_i \in M$.

In the light of the previous discussion we introduce a near-linear matching algorithm called *matching via weight-constrained label propagation in bipartite networks* (*MLPb*), outlined in Alg. 1. It incorporates the aforementioned premises and is based on the well-known *Label Propagation Algorithm* (*LPA*) for community detection in networks introduced by Raghavan et al. [20]. Because it is simple and fast, *LPA* provides an appropriate framework for a matching algorithm. However, whilst in its original formulation the algorithm returns a community membership array, *MLPb* outputs a hierarchy of increasingly simplified networks, which may be considered for solving multiple tasks, community detection included.

Algorithm *MLPb* takes as inputs the bipartite network at a coarsening level l , $G_l = (V^1 \cup V^2, E, \sigma, \omega)$ and the layer to be coarsened $j = \{1, 2\}$. Coarsening may be applied to one or both layers as *MLPb* operates on the layers independently to fit their corresponding parameters S_{max}^j and ζ^j . The remaining parameters are (for the corresponding layer to be coarsened) a target reduction factor $\rho = (0, 1.0] \subset \mathbb{R}$, an upper bound $\mu \in (0, \zeta] \subset \mathbb{R}$, and two stopping criteria, namely a maximum number of iterations

$T \subset \mathbb{Z}_+$ and a tolerance parameter $\tau \subset \mathbb{R}_+$ which specifies a minimum threshold on label updates. It returns the matching $M = \{V_k\}_{k=0}^K$, which can be stored as a union-find or as a simple array structure. $M(u)$ denotes a function that returns the super-vertex sV_i that includes vertex u , and $M(u) = sV_i$ is an operation that collapses u into the existing super-vertex sV_i .

MLPb propagates vertex labels throughout the network to create the super-vertices. A single label remains within a group of matched vertices that will collapse into a single super-vertex. Furthermore, to ensure independent handling of the layers, vertices can only match others in their set of 2-hop neighbors. This restriction implies, from the definition of bipartite networks, that adjacent vertices are not matched and potential matches of a vertex u can only be found in its own layer.

Input: $(G, j, \zeta, \mu, T, \tau)$

```

 $S_{max} = \lceil (1 + \mu) |V_0^j| / \zeta \rceil;$  1
 $M = \text{start each vertex as a singleton with a unique label};$  2
while  $T \neq 0$  and  $\tau < \text{swap}$  do 3
     $\text{swap} = 0;$  4
    for arbitrary  $u \in V_l^j$  do 5
         $Q = \text{start each label as score 0.0};$  6
        for  $v \in \Gamma_2(u)$  do 7
             $sV_i = M(v);$  8
            if  $\sigma(sV_i) + \sigma_l(u) \leq S_{max}$  then 9
                 $Q[sV_i] = Q[sV_i] + S_{wcn}(u, v);$  10
         $sV_{max} = \text{argmax}(Q);$  11
        if  $sV_{max} \neq sV_i$  then 12
             $\text{swap} = \text{swap} + 1;$  13
             $M(u) = sV_{max};$  14
            update the weights in  $\sigma;$  15
            if  $|M| \leq \zeta$  then 16
                 $\text{swap} = \tau;$  17
                break; 18
    decrease  $T;$  19

```

Return: M

Algorithm 1. *MLPb*: Matching via weight-constrained label propagation in bipartite networks.

The algorithm initially computes S_{max} and each vertex is initialized as a singleton with a unique label (lines 1–2). Vertices are visited in random order (line 5), and a randomly picked vertex u is matched within its 2-hop neighborhood with the vertex for which the number of weighted common neighbors is maximal (lines 7–11). As super-vertices are weight-constrained, a match is eligible if it does not yield an overloaded super-vertex (line 9). The matching process proceeds updating vertex labels and will be interrupted once the specified minimum number of vertices ζ is attained (line 16). Alternatively, the stopping conditions are given by input parameters T (the maximum number of iterations) and τ (a tolerance threshold on label updates): the current execution stops if either T has been reached or the number of label updates is smaller than or equal to tolerance τ .

The contraction algorithm is described in Alg. 2, and takes as input a matching M where matches are given by groups of vertices rather than vertex pairs. Groups of matched vertices are mapped into a *successor vector* and collapsed into a single super-vertex $sV_i \in G_{l+1}$ (lines 3–7). The *successor vector* will be accessed again in the uncoarsening phase to project the solution relative to its super-vertex sV_i . Any vertices not included in M are inherited by G_{l+1} (line 8). After the mapping, adjacent edges in E_l are collapsed into a single super-edge and added to E_{l+1} (lines 9–16), i.e. each pair $(u, v) \in E_l$ entails a successor (w, z) . If the successor is already in E_{l+1} , its weight is increased by $\omega_l(u, v)$; otherwise, a new edge (w, z) with weight $\omega_l(u, v)$ is inserted into E_{l+1} (lines 13–16).

Input: (G_l, M)
 $sucessor = \emptyset;$
 $G_{l+1} = (V_{l+1}, E_{l+1}, \sigma_{l+1}, \omega_{l+1}) = \emptyset;$
for $V' \in M$ **do**
 create new super-vertex sV to V_{l+1} with id n ;
 forall $u \in V'$ **do**
 $sucessor[u] = n$;
 $\sigma_{l+1}(sV) = \sigma_l(u) + \sigma_l(u)$;
Uncollapsed vertices are inherited by G_{l+1} ;
forall $(v, u) \in E_l$ **do**
 $w = sucessor[u]$;
 $z = sucessor[v]$;
 if $w \neq z$ or they are not a $sV \in E_{l+1}$ **then**
 if $(w, z) \in E_{l+1}$ **then**
 $E_{l+1} = \omega_{l+1}(w, z) + \omega_l(u, v)$;
 else
 $E_{l+1} = \text{new edge } (w, z) \text{ with } \omega_l(u, v)$;
Return: G_{l+1}

Algorithm 2. *Cb*: Contraction of a bipartite network.

A network coarsened with *MLPb* is, by definition, weighted, i.e., even if the initial network is unweighted the super-edges created in the coarsening will be assigned their edge count as weights. If not relevant for solving the target problem edge weights may be simply ignored.

Finally, algorithms *MLPb* and *Cb* can be integrated into a coarsening strategy. We propose two variations that differ in how the coarsening parameters are specified. Algorithm *MLPb_{CSL}* implements a standard formulation of the multilevel method, where the user directly sets the desired number of coarsening levels L and a reduction factor ρ . In contrast, in algorithm *MLPb_{CSV}* the user input is the desired size (number of vertices) expected in the coarsest network, ζ , and it builds the coarsening hierarchy so as to obtain a coarsest network with the user-specified size.

Alg. 3 describes *MLPb_{CSL}*. The inputs are, for each layer, the maximum number of levels $L = \{L^1, L^2\}$, the reduction factor $\rho = \{\rho^1, \rho^2\}$, the upper bound on super-vertex weights $\mu = \{\mu^1, \mu^2\}$ and the stopping conditions $T = \{T^1, T^2\}$ and $\tau = \{\tau^1, \tau^2\}$. Each layer is coarsened (line 4), level by level, by means of calls to matching algorithm *MLPb* (line 6), with ζ computed as a function of the current number of vertices and the aimed reduction factor (line 5). Finally, algorithm *Cb* (line 9) builds the coarser network from the matching M . This process continues until each layer is coarsened at the desired number of levels. Alternatively, Alg. 4 describes *MLPb_{CSV}*, where input parameters $\zeta = \{\zeta^1, \zeta^2\}$ specify the desired size of each layer.

Input: $(G_0, L, \rho, \mu, T, \tau)$
 $H = \emptyset;$
 $l = 0;$
while $l \leq L^j \forall j \in \{1, 2\}$ **do**
 for $j \in \{1, 2\} : l \leq L^j$ **do**
 $\zeta = \lceil |V_l^j| (1 - \rho^j) \rceil;$
 $M^j = MLPb(G_l, j, \zeta, \mu^j, T^j, \tau^j);$
 increase l ;
 $M = \cup M^j \forall j \in \{1, 2\};$
 $G_{l+1} = Cb(G_l, M);$
 $H.add(G_{l+1});$
Return: H

Algorithm 3. *MLPb_{CSL}*: Coarsening with number of levels and reduction factor as input parameters.

Input: $(G_0, \zeta, \mu, T, \tau)$
 $H = \emptyset;$
while $\zeta^j \leq |V_l^j| \forall j \in \{1, 2\}$ **do**
 for $j \in \{1, 2\} : \zeta^j \leq |V_l^j|$ **do**
 $M^j = MLPb(G_l, j, \zeta^j, \mu^j, T^j, \tau^j);$
 $M = \cup M^j \forall j \in \{1, 2\};$
 $G_{l+1} = Cb(G_l, M);$
 $H.add(G_{l+1});$
Return: H

Algorithm 4. *MLPb_{CSV}*: Coarsening with desired network size as input parameter.

In this case, the value S_{max} is static and the number of coarsening levels depends on how many iterations the algorithm executes before it stops. The algorithm applies the highest possible ρ at each level.

Fig. 7 illustrates the behavior of algorithm *MLPb_{CSL}* on a dense weighted bipartite synthetic network obtained setting $|V| = 2000$, $c = 5$ communities and noise level $n = 0.1$. Networks are drawn with a bipartite layout. Coarsening was executed setting $\rho = 0.5$ and $L = 5$. The initial network is shown in (a), whereas (b), (c), (d), (e) and (f) show, respectively, the coarsened networks at levels 1 to 5, with reduction factors of 25%, 50%, 75%, 90% and 95%.

Similarly to state-of-the-art coarsening strategies, algorithms *MLPb* are directly applicable in community detection tasks: each super-vertex in the coarsest network, illustrated in Fig. 7(f), may be interpreted as a community. In projecting the solution back to the original network the predecessor vertices of a super-vertex will be assigned to a community, identified by a corresponding color in Fig. 7(g). This strategy resembles bi-clustering or co-clustering solutions, which also define communities independently in each layer, unlike standard community detection algorithms in which communities obtained are formed by vertices from both layers. This feature may be useful in some contexts, for instance, when just one of the layers must be coarsened, as illustrated in Fig. 7(f), or the layers must be coarsened at different rates.

5. Parametric and complexity analysis

The *MLPb_{CSV}* implementation (Alg. 4) is more intuitive to use, as it suppresses input parameters L and ρ , replacing them with ζ , the target network size, which is more intuitive to specify. We conducted a parametric analysis to assess its convergence behavior. Analysis was conducted on synthetic networks of varying degree distributions and clustering coefficients, obtained employing a community generation model that mimics properties of real-world networks regarding noise, community structure and overlapping [21].

We considered a fixed value of parameter ζ for both layers, so the algorithm will iterate until $\zeta^1 = \zeta^2 = 10$, and created networks with the following settings of the generating model: vertex set sizes $|V| = |V^1| + |V^2|$ varying in the range [1000, 100,000] at increments of 300; noise levels n varying randomly in the range [0.1, 0.4]; numbers of communities given by $c = 0.01|V|$; network density d varying in the range [0.1, 0.5]. Overall, in the following analyses we considered 100 distinct configurations and replicated each configuration ten times to yield 1000 synthetic networks.

Fig. 8 shows the points depicting how many coarsening levels L have been created before the algorithm stops, as a function of network size $|V|$. Notice that, in general, L increases slowly as $|V|$

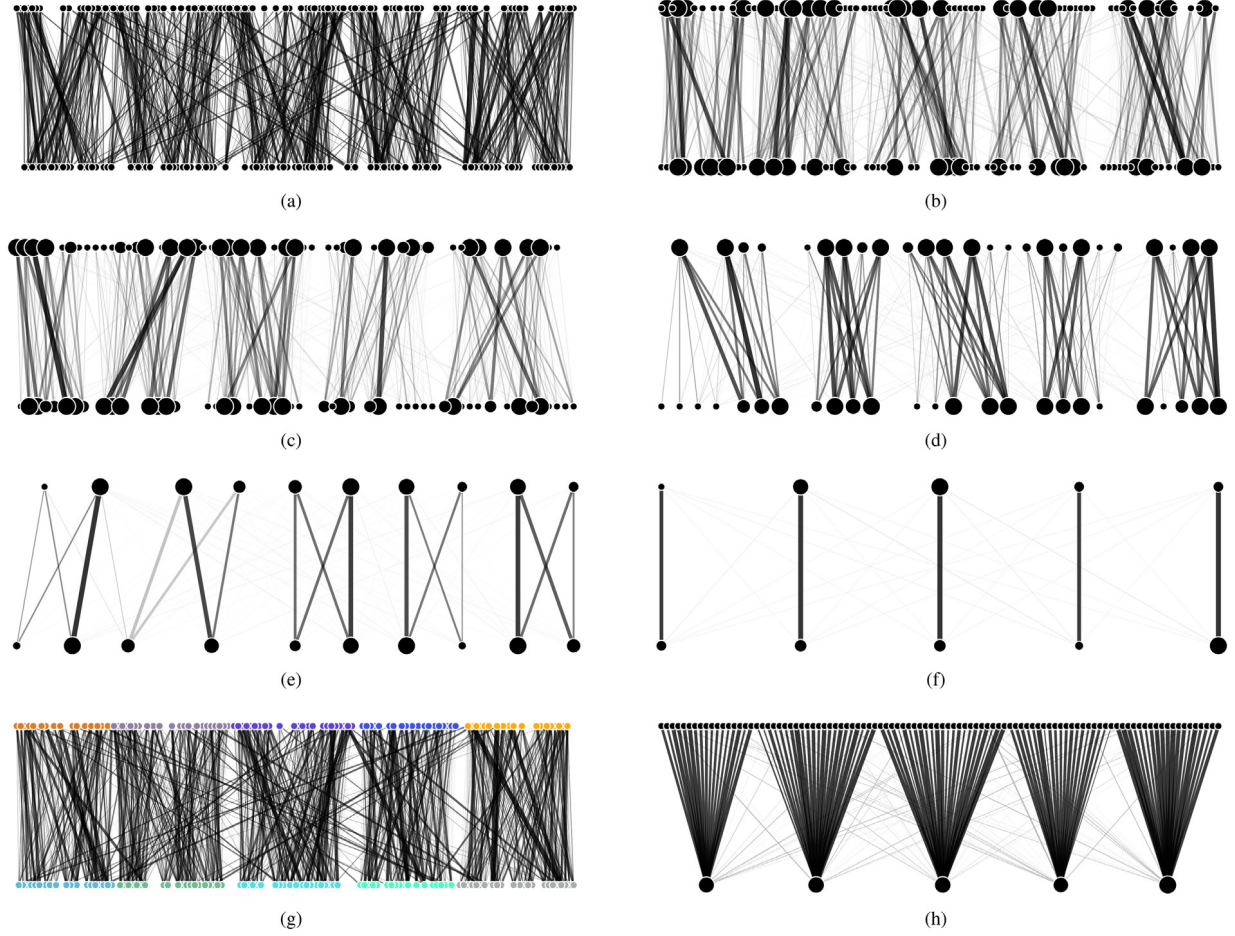


Fig. 7. A bipartite network coarsened at increasing reduction factors with the $MLPb_{CSL}$ algorithm. The original network is shown in (a) and the networks reduced at rates of 25%, 50%, 75%, 90% and 95% are shown, respectively, in (b), (c), (d), (e) and (f). If the super-vertices in (f) are considered as communities and projected back to the original network, the result is summarized in (g), with colors indicating the successor super-vertex. The plot in (h) shows the result of coarsening just one layer ($j = 2$) to five super-vertices.

increases. In summary, on smaller networks convergence occurs with L in the range $[2, 6]$, whereas on larger networks it occurs with L in the range $[6, 10]$.

Fig. 9 depicts how many iterations have been executed before the algorithm stops, also as a function of $|V|$. Each line refers to a particular setting of the tolerance parameter τ . In general, $T < 10$ iterations are required regardless of the value of τ . T increases quickly for network sizes up to 40,000 vertices, slowly for network sizes in the range $[40,000-80,000]$ vertices, and remains stable on larger networks. Higher values of τ speed up convergence.

Despite the simplification introduced by replacing input parameters L and ρ with parameter ζ , the need of setting the stopping conditions T and τ may be seen as a potential drawback of the $MLPb$ matching algorithms. Both are standard parameters of label propagation strategies and strategies are often employed to suppress them [20,22,23]. In the case of coarsening, one may define (i) the number of iterations T in terms of a percentage of the number of vertices matched in the first iteration; (ii) the tolerance parameter as equal to one, i.e. $\tau = 1$; and (iii) the upper bound on super-vertex weights as equal to the minimum size of layer j , i.e. $\mu^j = \zeta^j$. These are *ad hoc* values based on our experience with application of the method.

We also analyze the computational cost of coarsening considering the time complexity of both $MLPb$ (Alg. 1) and Cb . $MLPb$ is executed T times and requires few iterations to converge, hence

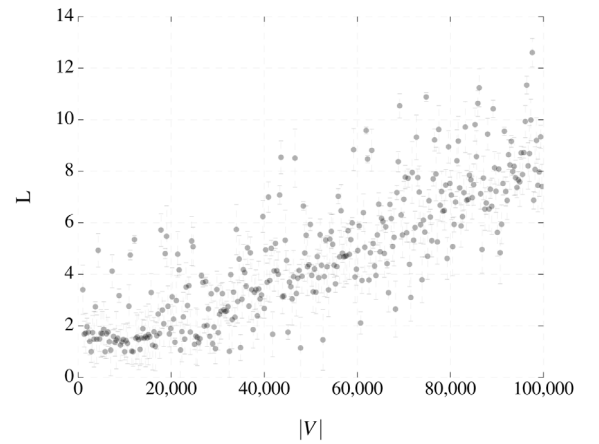


Fig. 8. Each point depicts the average number of coarsening levels L (and corresponding standard deviation errors) required for $MLPb_{CSV}$ to stop, as a function of network size $|V| = |V^1| + |V^2|$.

we consider T as a constant factor. The loop in line 7 iterates over $|V^j| - \zeta$, hence, in general, it takes time $\varphi = |V^j|/2$ with $\varphi \ll |V|$. The algorithm finds the h -hop neighborhood $\Gamma_h(u)$ of each selected vertex u (line 10) at a cost $\mathcal{O}(n\langle\kappa\rangle h)$, where $\langle\kappa\rangle$ is the

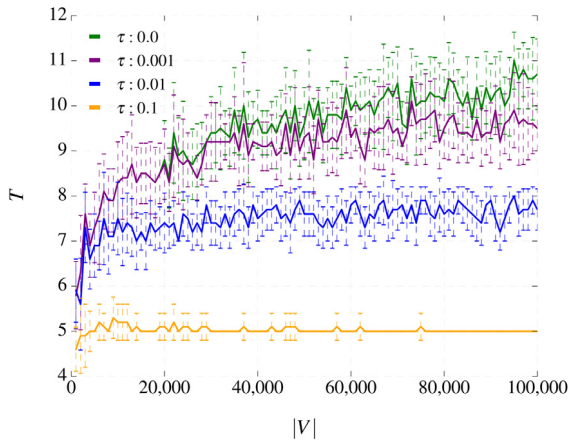


Fig. 9. Average values and corresponding standard deviation errors of the number of iterations T required for $MLPb_{CSV}$ to stop as a function of network sizes $|V|$, considering four distinct settings of the tolerance threshold parameter τ .

average network degree. In sparse networks $\langle \kappa \rangle \ll n$ and $m \approx n$. Restricting $h = 2$ yields notable computational savings, since Γ_2 can be reached in near-linear time. Therefore, in the average case, the time complexity is $\mathcal{O}(T\varphi(\kappa))$. In the worst-case, i.e., dense networks with high $\langle \kappa \rangle$ and balanced layers, the time complexity is $\mathcal{O}(|V|^2)$ with $|V^j| \approx |V|/2$. In the best-case, i.e., sparse networks with small $\langle \kappa \rangle$ and a small reduction factor the cost is $\mathcal{O}(\varphi)$ with $\varphi \ll |V|/2$.

The contraction Algorithm *Cb* (Alg. 2) first iterates over all collapsed vertices in $V' = \cup V_i \in M$ in order to create the *successor* data structure and the super-vertices. Next, each edge in E_l is selected to create the super-edges. Therefore, in the worst-case, i.e. $|V'| = |V|$, the asymptotic complexity is $\mathcal{O}(|V| + |E|)$. In the best-case, i.e. a sparse network and a small reduction factor or small ζ , it runs in $\mathcal{O}(|V'| + |E|)$ with $|V'| \ll |V|$ and $|E| \approx |V|$, then complexity is $\mathcal{O}(|V|)$.

6. Experimental results and discussion

We conducted three experimental studies to assess the proposed coarsening algorithms:

1. A first study compared the performance of $MLPb_{CSL}$ with three state-of-the-art coarsening algorithms to optimize an expensive community detection algorithm. The goal is to evaluate the quality the topological structure of the coarsened representations obtained.
2. A second study employed $MLPb_{CSV}$ as a community detection algorithm, comparing results with those obtained with three popular and fast community detection algorithms. The goal is two-fold: evaluate to which extent the super-vertices created are representative of the network community structures; and verify the algorithm's flexibility to handle different tasks.
3. Finally, a third study employed $MLPb_{CSV}$ in association with a force-directed network drawing algorithm for purposes of visualizing major topological structures in large bipartite networks. The goal is to assess to which extent the coarsening of the networks can faithfully preserve and summarize the global properties of bipartite networks with distinct topological characteristics.

Experiments were executed on a Linux machine with an 8-core processor with 3.7 GHz CPU and 64 GB main memory. All

algorithms were implemented in Python with the *Igraph*¹ and *Networkx*² libraries.

6.1. Benchmark optimization problem

Research efforts on the multilevel method were strongly motivated by expensive optimization algorithms, most notable community detection, which established itself as a benchmark problem in this context. Therefore, we assess the proposed coarsening strategy in the context of multilevel community detection on bipartite networks.

Larremore et al. [13] formulated a bipartite stochastic block model algorithm to detect communities in bipartite networks (*biSBM*³⁴). *BiSBM* relies on the Kernighan–Lin criterion as a heuristic to infer the ideal number of communities, and has competitive performance relative to state-of-the-art methods. However, it is too expensive to be executed on large-scale networks.

We thus considered *biSBM* as the target algorithm for a comparative assessment of the new $MLPb_{CSL}$ coarsening algorithm. The goal is to investigate to which extent the coarsening processes executed with $MLPb_{CSL}$ and with other state-of-the-art coarsening algorithms for bipartite networks can yield solutions statistically equivalent in quality to the standard *biSBM*, used as baseline, while increasing its scalability. Our experimental study compared $MLPb_{CSL}$ with coarsening algorithms OPM_{hem} , $RGMB$ and GMB , discussed in Section 2. We assessed the runtime and quality of the reduced networks obtained from each algorithm. Henceforth, the multilevel versions of *biSBM* will be identified as $MLPb_{CSL}+biSBM$, $GMB+biSBM$, $RGMB+biSBM$ and $OPM_{hem}+biSBM$.

A set of 60 parameter configurations were considered to generate synthetic weighted bipartite networks with distinct community structures [21], as follows: $|V| = 1000$ with $|V^1| = |V^2|$, n within the range $[0.0025, 0.3]$ at increments of 0.0025, and the number of communities $c = 10$. Finally, the network density d varied randomly in the range $[0.1, 0.8]$ at increments of 0.1. Each network setting was replicated 10 times, resulting in 600 synthetic networks. Two performance measures were considered: normalized mutual information (*NMI*) and runtime (seconds). The *NMI* index quantifies the quality of the disjoint clusters inferred [24] and is often employed to compare community detection algorithms.

Fig. 10 shows the *NMI* accuracy values as a function of the amount of noise in the synthetic networks. Fig. 10(a) shows results from the baseline and its multilevel version at one level of coarsening (i.e., $L = 1$) with $\rho = 0.5$ (i.e., reducing by half the number of vertices) for all layers. In general terms and considering the variability measured by the standard deviation, the best performances were achieved by the baseline and the multilevel version $MLPb_{CSL} + biSBM$. However, considering the noise parameter varying in the range in $[0.25, 0.3]$, the baseline yielded the best average performance in all networks. *NMI* values for $GMB + biSBM$, $RGMB + biSBM$ and $OPM_{hem} + biSBM$ decrease more quickly than the baseline and $MLPb_{CSL} + biSBM$, as the noise levels increase. Furthermore, considering the noise in the range $[0.15, 0.3]$, $GMB + biSBM$, $RGMB + biSBM$ and $OPM_{hem} + biSBM$ yielded the lowest average performance in all networks. In this case, the reduced network representations created with $MLPb_{CSL}$ proved more robust than those obtained with GMB , $RGMB$ and OPM_{hem} .

Fig. 10(b) shows the results of the baseline and its multilevel versions considering $L = 2$ and $\rho = 0.5$ for all layers. For noise

¹ Available from <http://igraph.org/python/>.

² Available from <https://networkx.github.io/>.

³ C++: <http://danlarremore.com/bipartiteSBM/>.

⁴ Python interface: https://github.com/junipertcy/det_k_bisbm.

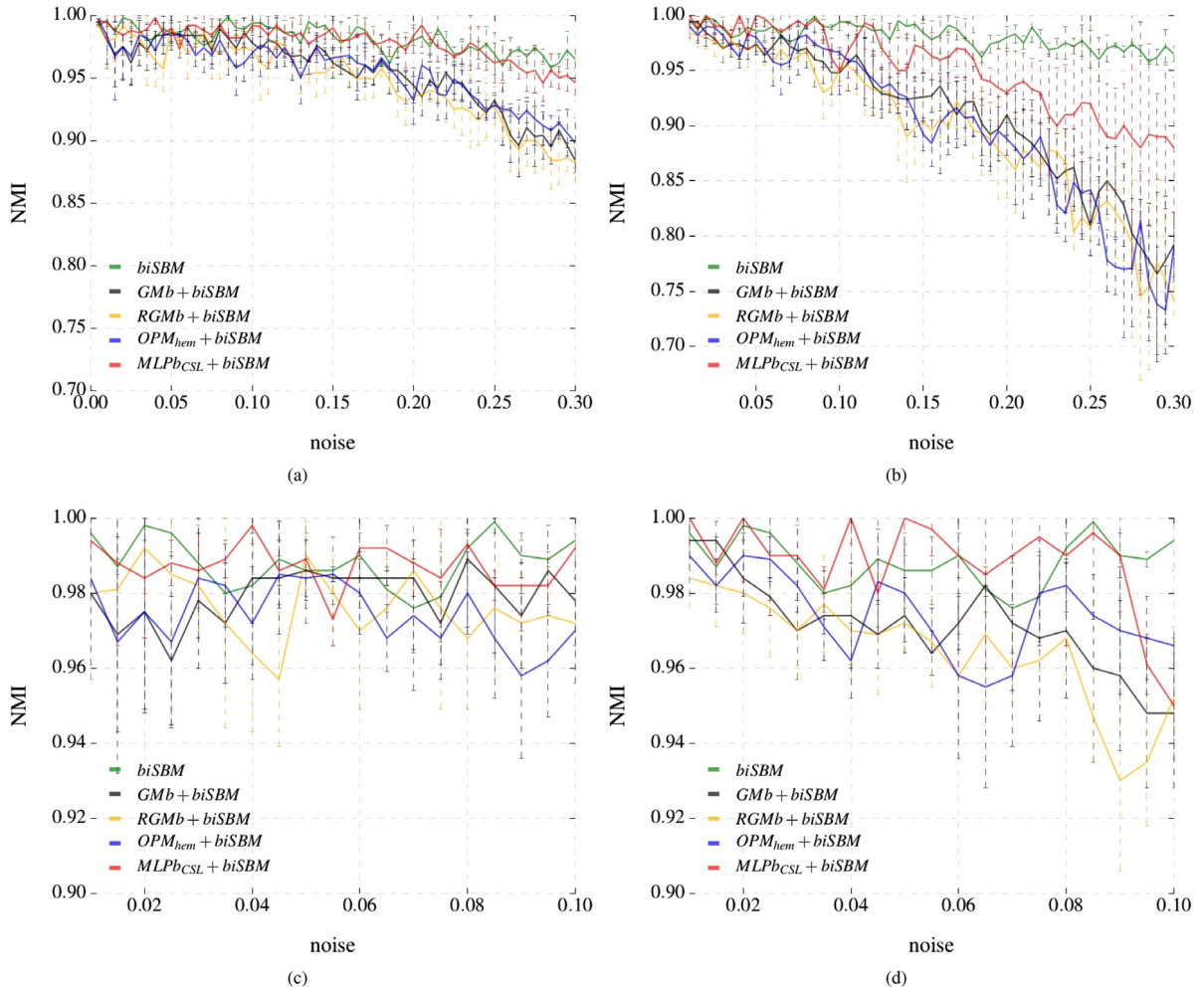


Fig. 10. NMI accuracy values of *biSBM* and its multilevel versions as a function of the noise levels for 60 synthetic networks with different properties. (a) NMI values achieved by the baseline and its multilevel optimization at one level of coarsening; (b) zoomed-in view of the plot for noise levels in the range [0.0025, 0.1]; (c) NMI values at two levels of coarsening; (d) zoomed-in view of the plot for noise levels in the range [0.0025, 0.1].

levels within the range [0.0025, 0.15] the baseline and *MLPb_{CSL} + biSBM* yielded the best performances. However, for noise levels above 0.15 the baseline yielded the best average performance. In this case, *Gmb + biSBM*, *RGMb + biSBM* and *OPM_{hem} + biSBM* show similar performance and high sensitivity to the noise level (NMI values decrease fast as noise increases), whereas results from *MLPb_{CSL} + biSBM* are closer to the baseline, i.e., the decrease in NMI values is slower compared to other coarsening algorithms, suggesting greater robustness to noise. Furthermore, with $L = 2$ results from all coarsening algorithms exhibit high standard deviations, suggesting instability. This might be explained by the sharp decrease in the number of potential super-vertices at each coarsening level.

Figs. 10(c) and 10(d) illustrate the zoomed-in plots focusing on networks with low noise levels, in the range [0.0025, 0.1]. One notices a small loss in accuracy, even at two levels of coarsening. Moreover, the standard deviations of the values suggest the difference between the baseline and the multilevel versions is not significant.

A Nemenyi post-hoc test [25] was applied to the results in Fig. 10 to verify statistical significance of the observed differences in algorithm performance. Results are shown in Fig. 11, in which (a) and (b) depict, respectively, the ranks for $L = 1$ and $L = 2$, and (c) and (d) depict the average ranks relative to the zoomed-in plots. The critical difference (CD) is indicated at the top of each diagram and algorithms' average ranks are placed in the

horizontal axes, with the best ranked to the left. A black line connects algorithms for which there is no significant performance difference. According to the Nemenyi statistics the critical value for comparing the average-ranking of two algorithms at the 95 percentile is 0.97 for (a), 0.99 for (b), 1.73 for (c) and 1.76 for (d).

Let us consider the outcome of the post-hoc test for $L = 1$, i.e., when the number of vertices n has been reduced by a factor of two, shown in Fig. 11(a). Baseline *biSBM* ranked best, followed by *MLPb + biSBM* and then the group *Gmb + biSBM*, *RGMb + biSBM* and *OPM_{hem} + biSBM*. Furthermore, even though it performs better than the other algorithms, performance differences between *biSBM* and *MLPb_{CSL} + biSBM* are not statistically significant. Similarly, for $L = 2$ (Fig. 11(a)), *biSBM* ranked first, with no statistically significant difference observed relative to *MLPb_{CSL} + biSBM*.

Interestingly, in the networks with noise levels in the range [0.0025, 0.1], for both $L = 1$ and $L = 2$, algorithm *MLPb_{CSL} + biSBM* ranked best, reinforcing that *MLPb_{CSL}* can filter the solution space while preserving important topological features. For $L = 2$ both *MLPb_{CSL} + biSBM* and *biSBM* ranked first. In contrast, the accuracy of *Gmb*, *RGMb + biSBM* and *OPM_{hem} + biSBM* deteriorates, mainly when $L = 2$, indicating that coarsening at this level does not necessarily preserve the original topological properties and is likely to blur community boundaries. The impact of the choice of L depends on network size, i.e., variations in algorithm accuracy are likely to decrease as network sizes increase, hinting that adopting

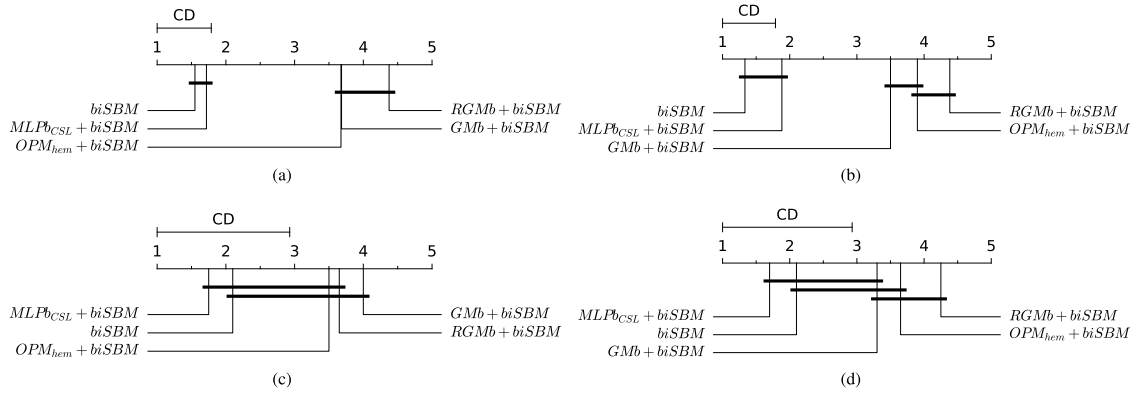


Fig. 11. Nemenyi post-hoc test for *biSBM* and its multilevel versions: (a) shows rank values achieved by the baseline and its multilevel optimization with one level of coarsening and (c) illustrates the rank values for the zoomed-in plot with noise in the range in $[0.0025, 0.1]$; (b) shows the rank values with two levels of coarsening and (d) illustrates the rank values for the zoomed-in plot with noise in the range of $[0.0025, 0.1]$.

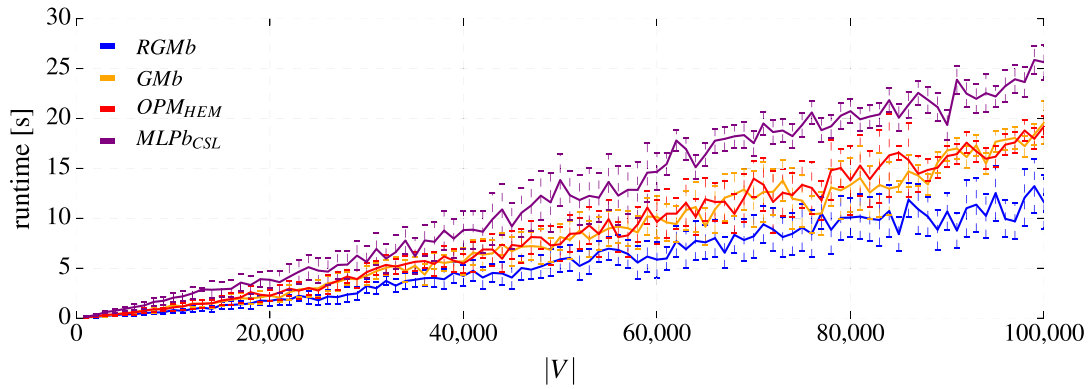


Fig. 12. Runtime (absolute value, in seconds) of four coarsening algorithms to build a coarsest representation for 1000 synthetic networks.

more aggressive coarsening factors might be a successful strategy when handling larger networks.

We also assessed the scalability of the multilevel versions of algorithm *biSBM*, in terms of the average total time spent by each algorithm. Considering all algorithms, the total experimental runtime was 112,198 s, or nearly 31 h. Fig. 13 shows how each algorithm contributed to the total time, in both absolute values (y-axis) and relative values (percentages shown on top of the bars). In the best case, *MLPb_CSL* + *biSBM* with $L = 2$ executed in 2808 s (0.78 h or 2.5% of the total time), whereas the standard *biSBM* executed in 57,062 s (nearly 15.8 h, or 50.9% of the total time), which is 20 times slower than its multilevel implementation.

Additionally, We assessed the impact of each multilevel phase, i.e., coarsening, solution finding executing *biSBM* on the coarsest network, and uncoarsening, on execution time. Fig. 14 shows the relative contributions of each phase to the total time for processing all networks. The coarsening phase consumed nearly 0, 2% of the average total time, whereas the execution of *biSBM* consumed the remaining 98, 8%. The time spent in the uncoarsening phase was thus negligible. These results provide empirical evidence on the negligible impact of the coarsening and uncoarsening phases on the scalability of the multilevel method, thus reinforced as a powerful strategy to scale network optimization algorithms.

Finally, to compare their execution times regardless of a particular application, we executed the coarsening algorithms on 1000 synthetic weighted bipartite networks with community structure, generated with the following parameter configurations [21]: $|V| = |V^1| + |V^2|$ varying in the range $[500, 100,000]$ at increments of 500; noise level n varying in the range $[0.1, 0.4]$

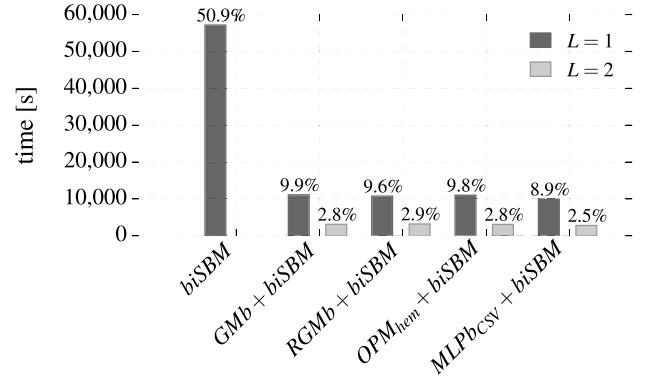


Fig. 13. Contribution of each algorithm to the total time spent on the experiments (absolute and relative values). The total runtime (average of 10 executions) was 112,198 s, or 31 h.

at increments of 0.05; number of communities $c = 0.01|V|$, and network density $d = 0.3$.

Fig. 12 shows the absolute runtime (in seconds) of each coarsening algorithm in each network. Values refer to average times relative to 10 executions; each algorithm iterated up to 10 coarsening levels. One notices a linear correlation between runtime and number of vertices $|V|$. *RGMb* yielded the shortest time and *MLPb_CSL* the longest, whereas algorithms *OPM_HEM* and *Gmb* show very similar behavior regarding execution times.

Besides network size, algorithm *MLPb_CSL* depends on the number of iterations required prior to convergence. In small networks, few labels updates occur at each iteration, whereas in larger

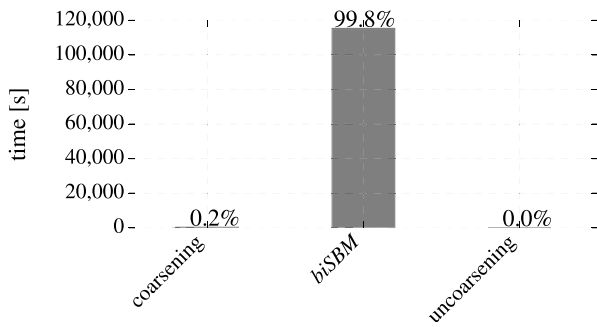


Fig. 14. Contribution of each multilevel phase, considering all networks and algorithms (absolute and relative values relative to total execution time).

networks the probability of label updates is higher. Therefore, increasing $|V|$ directly impacts algorithm convergence and its asymptotic complexity. Indeed, according to Fig. 12, differences are small in execution times of $MLPb_{CSL}$ relative to the other algorithms in networks with up to 40,000 vertices, and increases gradually as $|V|$ increases.

One also observes in Fig. 12 that $MLPb_{CSL}$ has similar asymptotic complexity but higher execution time compared to the other algorithms. Interestingly, in the first experiment the lowest execution time was obtained with $MLPb_{CSL} + biSBM$. Indeed, $MLPb_{CSL}$ builds more robust networks at the cost of a small loss in the topological structure. Therefore, $biSBM$ requires less iterations to identify the best number of communities, which directly affects its execution time.

The previous analysis indicates that $MLPb_{CSL}$ yielded superior results compared to the state-of-the-art algorithms OPM_{hem} , $RGMB$ and GMB that operate on vertex pairs. It is also evident that, albeit they are fast, these algorithms are limited in their capability of preserving and revealing the underlying community structures in the networks. In the following study we investigate the behavior of $MLPb_{CSL}$ in the community detection problem. Our goal is to verify the quality of the super-vertices computed and the algorithm's adaptability to handle complex tasks.

6.2. Community detection

We assessed the behavior of $MLPb_{CSV}$ in the community detection problem, comparing it with three well-established algorithms known to be fast and effective,⁵ namely LP [20,26], $Louvain$ [27] and $Fastgreedy$ [28]. The motivation for this empirical study is to assess the quality of the coarsening obtained with the algorithm, verifying whether the super-vertices it creates are comparable to the communities identified by reputed network partitioning algorithms from the literature. Since highly quality communities are expected to group vertices with similar properties, function or meaning, expressed as neighborhood or topological patterns, abstracting super-vertices as communities provides a reliable mechanism to verify their representativeness.

$Louvain$ is a multilevel algorithm that relies on Newman's modularity maximization to infer the best community structure; LP is a label propagation algorithm that maximizes modularity; and $Fastgreedy$ is a hierarchical agglomeration algorithm based on modularity maximization. The number of communities is not specified *a priori* in these algorithms, therefore, they have been evaluated considering the number of communities they inferred. The selected algorithms are fast, with asymptotic complexity close to the $MLPb$, memory-efficient and known to yield meaningful community structures in practice. Moreover, reliable and

Table 1

Properties of the largest connected component of the real-world bipartite networks employed in the experimental evaluation.

Dataset	$ V^1 $	$ V^2 $	$ E $	$\kappa(G)$	$\zeta(G)$
Unicodelang	248	610	1,249	2.91	0.43
Moreno	754	509	1,377	2.18	0.42
leeVis	3,919	2,378	8,868	2.82	0.35
N-reactome	8,788	15,433	41,087	3.39	0.32
Condmatt	13,861	19,466	53,628	3.21	0.34
Dbpedia	54,909	19,866	98,895	2.64	0.45

optimized Python implementations are available in the Networkx library.⁶

$Louvain$, $Fastgreedy$ and LP have been applied to the one-mode projections (see Fortunato [29]) of the tested bipartite networks following the guidelines given in Chessa et al. [30], Alzahrani and Horadam [31] and Cann et al. [32]. Furthermore, considering that the three algorithms rely on modularity maximization and $MLPb_{CSV}$ relies on assessing neighborhood similarity, in order to ensure a fair comparison we adopted the well-known quality metric Conductance (C), rather than modularity. Conductance assesses the separability of a community from the rest of the network [33], measured by the ratio between the number of its internal versus external edges. C values are normalized in the range $[0, 1]$, where 1 is the optimal score. We also measured runtimes of the algorithms.

Employing $MLPb_{CSV}$ for community detection is straightforward. It suffices to coarsen the network to a defined number of super-vertices (using the CSV strategy, Alg. 4), and assign to each super-vertex a unique label. Super-vertex labels are then assigned to their corresponding predecessor-vertices at each coarsening level along the projection phase, until the original network. Finally, vertices assigned the same label define the resulting communities (revisit Fig. 7).

In assessing the algorithm we considered six real bipartite networks representative of diverse topological properties, available at KONECT.⁷ (the Koblenz Network Collection). We took the largest connected component of each network, for which the size of each layer $|V^1|$ and $|V^2|$, the number of edges $|E|$, the average degree $\kappa(G)$, and average clustering coefficient $\zeta(G)$ are detailed in Table 1. The upper bound parameter was set as $\mu = 0.3$ to allow for slightly unbalanced super-vertices.

The number of desired communities k determines the number of super-vertices in the coarsest network, i.e. $\zeta = k$. We used the following settings: $k = 5$ for Unicodelang and Moreno, $k = 10$ for leeVis, $k = 20$ for N-reactome, $k = 35$ for Condmatt, and $k = 40$ for Dbpedia, i.e., we adopt higher values for k as network sizes increase.

Table 2 summarizes the separability scores C relative to each algorithm (average values over 10 executions). Both $MLPb_{CSV}$ and $Louvain$ yielded the best C values on 3 out of the 6 networks. LP performed best on a single network, while $Fastgreedy$ did not perform best on any of them.

A Friedman test and a Nemenyi post-hoc test at the 95% confidence level [25] have been applied to the results in Table 2 to verify statistical significance in the observed differences in algorithm performance; results are shown in Fig. 15.

The final rows in Table 2 inform the average rank of the Friedman statistical test and the corresponding rank of the algorithm. According to the Nemenyi statistics, the critical value for comparing the mean-ranking of two different algorithms at 95 percentile is 1.91, i.e. significant differences are above this

⁵ Parallel or distributed algorithms have not been considered.

⁶ <https://networkx.github.io/>.

⁷ <http://konect.uni-koblenz.de/>.

Table 2

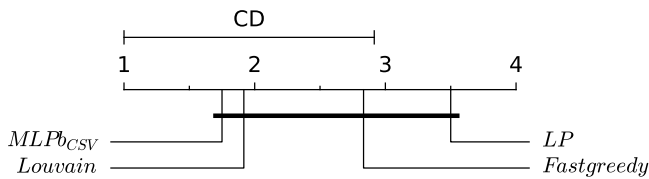
C scores, average rank and rank of the four algorithms in 6 real-world networks. The values correspond to average values over 10 executions.

Dataset	Algorithm			
	<i>MLPb_{CSV}</i>	<i>Louvain</i>	<i>LP</i>	<i>Fastgreedy</i>
Unicodelang	0.471 (3rd)	0.439 (4th)	0.571 (1st)	0.524 (2nd)
Moreno	0.934 (1st)	0.894 (2nd)	0.618 (4th)	0.874 (3rd)
leeVis	0.829 (1st)	0.829 (1st)	0.522 (4th)	0.811 (3rd)
Nreactome	0.561 (1st)	0.547 (2nd)	0.372 (4th)	0.408 (3rd)
Condmatt	0.812 (2nd)	0.819 (1st)	0.527 (4th)	0.802 (3rd)
Dbpedia	0.831 (2nd)	0.882 (1st)	0.518 (4th)	0.829 (3rd)
Average	0.74	0.521	0.735	0.708
Average rank	1.75	1.917	3.500	2.833
Rank	1st	1st	3rd	2nd

Table 3

Execution times (seconds) of each algorithm on each network. We disregarded the time spent on read, write and one-mode projection operations.

Dataset	Algorithm			
	<i>MLPb_{CSV}</i>	<i>Louvain</i>	<i>LP</i>	<i>Fastgreedy</i>
Unicodelang	0.196	0.3721	0.1861	1.3123
Moreno	0.0544	0.1263	0.1161	0.2994
leeVis	0.5091	1.2587	0.9506	10.1857
N-reactome	12.386	41.3638	34.5531	148.8097
Condmatt	4.3236	10.1867	10.1815	418.1561
Dbpedia	13.4365	54.9436	38.4371	1976.5222
Total	35.345	102.662	87.552	2,559.533

**Fig. 15.** Critical difference diagram of the results in Table 2.

value. The differences observed are not statistically significant, as indicated by the bold line in Fig. 15. Nevertheless, *MLPb_{CSV}* was ranked first followed by *Louvain*, *Fastgreedy* and *LP* in last. We conclude that *MLPb_{CSV}* achieved competitive accuracy with *Louvain*, *Fastgreedy* and *LP*.

We also computed algorithm execution times on each individual network, shown in Table 3, where values refer to the average values over 10 executions in each network. *MLPb_{CSV}* runtimes were the lowest in all networks, with speedups of around 2.5 relative to *LP*, 3 relative to *Louvain*, and 72.5 relative to *Fastgreedy*.

MLPb_{CSV} is fast because it operates directly in the bipartite network and separately on each layer, splitting the search space. Operating on one-mode projections, in contrast, implies in increasing considerably the number of edges handled by *Louvain*, *LP* and *Fastgreedy*, even if the number of vertices has been reduced by half. Moreover, optimizing Newman's modularity is computationally more expensive than just counting common 2-hop neighbors between vertices. *LP* often requiring many iterations before convergence as it also optimizes modularity locally on all vertices.

The results obtained allow us to conclude that *MLPb_{CSV}* is faster than the algorithms *Louvain*, *Fastgreedy* and *LP* and its super-vertices are representative of the community structures found by these algorithms. Furthermore, although the algorithm has not been designed for community detection its application to this task is straightforward.

6.3. Network coarsening for visualization

Network visualization deals with the problem of creating visual representations of networked data that support user-driven exploratory investigation. Visualizations should have reduced visual complexity and be aesthetically pleasing, and are expected to assist users in gaining insight on the global and local behavior of a dataset and improve their comprehension of the data.

The popular node-link layouts face severe limitations [34]. First, the drawings quickly become crowded as the number of vertices and edges easily exceed the number of display pixels available [34]. Overlapping of the graphical elements depicting vertices and edges impair readability and user interaction. The high computational cost that prevents real-time rendering of large networks is an additional limitation of classic force-directed network drawing algorithms [35–37]. These limitations are aggravated on bipartite networks, in which issues due to sparsity and complexity of topological structures can be even more severe than in unipartite networks.

Multilevel methods have been successfully employed to compute node-link layouts of unipartite networks [38]. Other studies combine aggregation and clustering, which is an approach that resembles coarsening in that they also rely on creating simplified representations of the full network [39,40]. However, we are not aware of previous efforts of employing the multilevel method for visualizing bipartite networks.

As network coarsening should preserve the network structures relevant for problem solving, we hypothesize it can provide a useful strategy to depict simplified views of large networks for visualization purposes. In this section we report an empirical investigation of this assumption. In particular, we illustrate how coarsening with *MLPb_{CSV}* creates summarized node-link representations that preserve the major characteristics of an original network whilst reducing the amount of information displayed. We show several illustrative examples using synthetic and real-world networks with diverse topological properties. All the network drawings have been computed with the force-directed algorithm introduced by Eades [41] and later improved by Kamada and Kawai [42] and Fruchterman and Reingold [43]. We used the Igraph Library⁸ implementation and rendered drawings with a resolution of 800 × 800 pixels.

Fig. 16(a) depicts a synthetic sparse bipartite network with five balanced communities formed by 2000 vertices ($|V^1| = |V^2| = 1000$) and with extensive noise, i.e., many edges between communities. Due to its size, sparsity and noise, community boundaries are not clearly visible. Communities can be better distinguished using colors, as in Fig. 16(b), or the colors can be combined with a strategy of displacing the communities apart, as in Fig. 16(c). These are common strategies for rendering network drawings in several network analysis software, such as NetworkX,⁹ Igraph,⁸ Gephi¹⁰ or Cytoscape.¹¹ Nonetheless, the problems remain that computing the layout is extremely slow and the resulting layout shows severe overlapping of graphical elements. Such drawings are of limited use for purposes of interactive visualization.

MLPb was executed to obtain a coarsened network with $|V| = 200$ ($|V^1| = |V^2| = 100$), drawn in Fig. 16(d). The five communities and their boundaries are very explicit in the reduced network. Outlier and isolated vertices are easily identifiable and constrained within the display limits. Furthermore, cross-edges

⁸ <https://igraph.org/>.

⁹ <https://networkx.github.io/>.

¹⁰ <https://gephi.org/>.

¹¹ <https://cytoscape.org/>.

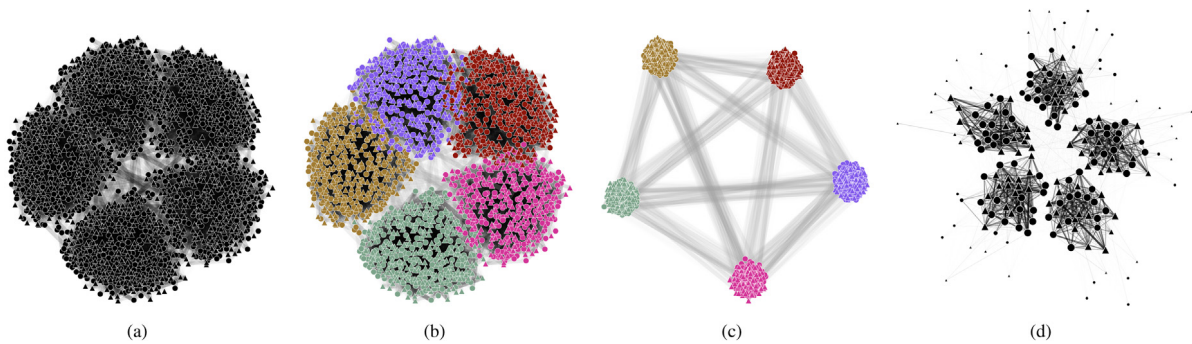


Fig. 16. Multiple layouts of a synthetic sparse bipartite network with $|V| = 2000$ vertices ($|V^1| = |V^2| = 1000$) distributed into 5 balanced communities with extensive noise. All layouts have been computed with the force-directed algorithm; in (b) and (c) the communities are identified with colors. (d) depicts the layout of the network coarsened to $|V| = 200$ vertices with $MLPb_{CSV}$.

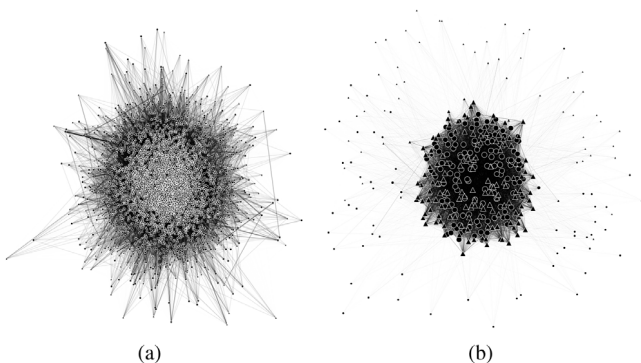


Fig. 17. Synthetic bipartite network drawings obtained with a force-directed algorithm: (a) shows the original network with no community structure, regular degree distribution and $|V| = 5000$; (b) illustrates a coarser representation obtained with $MLPb_{CSV}$, with $|V^1| = |V^2| = 200$.

between communities and the two types of vertices can be distinguished. The coarser representation reduces the clutter and the amount of information displayed, whilst still conveying the essential global structure of the full network.

This behavior can be explained by the data structure created by the contraction algorithm. The edge weights in a neighborhood (intra-community edges) increase at each coarsening level, unlike the weights of edges crossing the neighborhoods (inter-community edges). Furthermore, the weight of a super-vertex depends on the number of predecessor-vertices, thus lower-weights can indicate a small or less significant neighborhood. Therefore, the data structure emphasizes the community patterns in the network layout, since it is used by the force-directed algorithms in computing the layout.

As another example, Fig. 17(a) shows a synthetic network with regular degree distribution and no community structure, with $|V| = 5000$.

Fig. 17(b) depicts a coarsened version obtained executing $MLPb_{CSV}$ with $|V^1| = |V^2| = 400$. Drawing the original network takes around 1000 s, against 10 seconds to draw the coarser network. This example illustrates that $MLPb_{CSV}$ does not introduce topological structures that are not present in the original network. Rather, the drawing of the reduced network is actually more effective in revealing outliers such as the highest and lowest degree vertices. For purposes of an making an initial assessment of its topological structure, it is more convenient to observe a coarsened version of the network than waiting a long time for a drawing of the full model that reveals limited information and does not support interaction.

Indeed, smooth user navigation is a critical challenge in visualization of large networks, as users expect to grasp information relative to both local and global contexts [34]. In these scenarios, $MLPb_{CSV}$ could support interactive visualization of large-scale bipartite networks by means of navigation over the hierarchy of coarsened networks, which would be kept in an appropriate data structure. A super-vertex could be uncoarsened into its predecessor-vertices on demand, i.e., its content is “exploded” in response to a user interaction. Another possible strategy would be to redraw a selected neighborhood at a user-defined coarsening level, with the coarsening computed on the fly.

We further illustrate the behavior of $MLPb_{CSV}$ coarsening in two well-known real-world bipartite networks. In the corresponding network drawings, vertex weight is mapped to size and color of the graphical markers depicting super-vertices. We adopted a black-to-red color scale, so that bright red markers denote high-weight super-vertices. Moreno Crime is a social network with scale-free behavior relating 754 crime suspects or victims with 509 crimes (Fig. 18); N-reactome is a biological network with modular topology with 8788 proteins and 15,433 interactions in the species *Homo Sapiens* (Fig. 19). These networks are available in the Koblenz Network Collection (KONECT).¹²

Fig. 18(a) shows the original Moreno Crime network and Figs. 18(b), 18(c) and 18(d) illustrate it at three gradually increasing levels of coarsening, respectively with $|V| = 800$ ($|V^1| = 500$ and $|V^2| = 300$), $|V| = 500$ ($|V^1| = 300$ and $|V^2| = 100$) and $|V| = 150$ ($|V^1| = 100$ and $|V^2| = 50$). This network’s degree distribution follows the power-law relationship characteristic of scale-free networks. This behavior is reasonably preserved in the drawings of the coarsened models, up to a certain level, e.g., the presence of both hubs and low-degree vertices is evident in Figs. 18(b) and 18(c).

Fig. 19(a) shows the N-reactome network and Figs. 19(b) to 19(h) illustrate it at eight increasing coarsening levels, with the number of vertices set respectively to $|V| = 18,000$ ($|V^1| = 6000$ and $|V^2| = 12,000$), $|V| = 10,000$ ($|V^1| = |V^2| = 5000$), $|V| = 5000$ ($|V^1| = |V^2| = 2500$), $|V| = 2000$ ($|V^1| = |V^2| = 1000$), $|V| = 1600$ ($|V^1| = |V^2| = 800$), $|V| = 1000$ ($|V^1| = |V^2| = 500$), $|V| = 200$ ($|V^1| = |V^2| = 100$) and $|V| = 100$ ($|V^1| = |V^2| = 50$). This network has a modular topology with four noticeable modules. Again, the original topological structure and overall organization are preserved up to a certain level. At more advanced coarsening levels the original topological properties are lost as more vertices are collapsed into super-vertices, as observed mainly from 19(e) onwards. Nonetheless, three well-defined modules are still visible even in Figs. 19(f) and 19(g), and the essential global topology is still noticeable in Fig. 19(h).

¹² <http://konect.uni-koblenz.de/>.

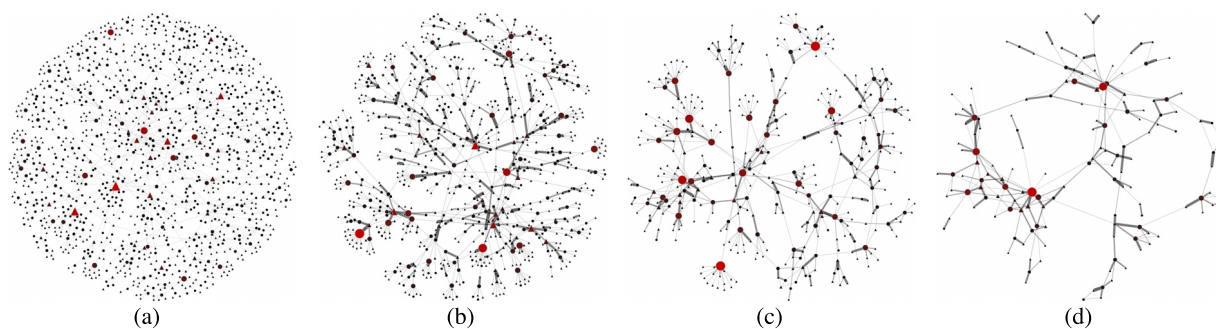


Fig. 18. Moreno Crime social network with scale-free behavior, which relates 754 criminal case suspects or victims with 509 crimes. (a) shows the original network, (b), (c) and (d) illustrate it at three increasing levels of coarsening, with $|V| = 800$, $|V| = 400$ and $|V| = 150$, respectively.

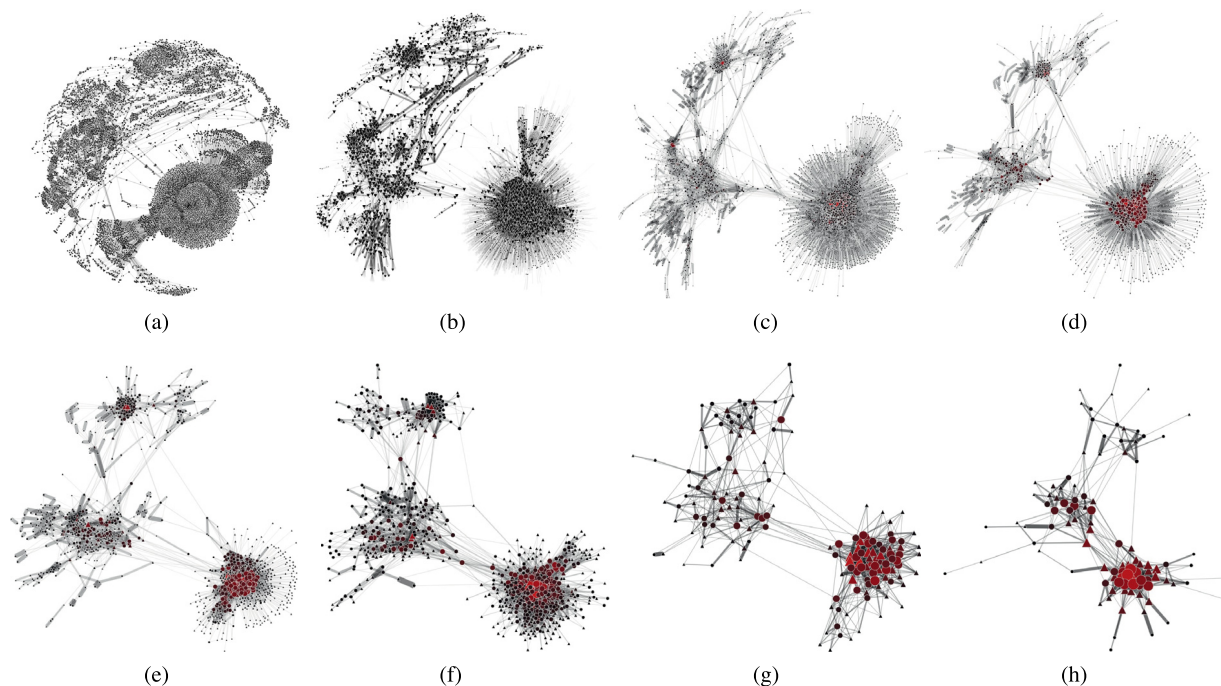


Fig. 19. N-reactome biological network with modular topology, depicting 8788 proteins and 15,433 interactions in the species *Homo Sapiens*. (a) shows the original network and (b), (c), (d), (e) (f), (g) and (h) illustrates the network at eight increasing levels of coarsening, with $|V| = 12,000$, $|V| = 10,000$, $|V| = 5000$, $|V| = 2000$, $|V| = 1600$, $|V| = 1000$, $|V| = 200$ and $|V| = 100$, respectively.

For comparison purposes we show in Fig. 21 drawings of the N-reactome network coarsened with the *RGMb* pairwise matching algorithm. Setting parameters ρ (reduction factor) and L (number of coarsening levels) is not intuitive and we executed the algorithm several times with alternative tentative settings before we succeeded in obtaining a coarsened network with around 100 vertices. Setting $\rho = 0.5$, 11 coarsening levels were necessary to yield $|V^1| = 68$ and $|V^2| = 56$ (Fig. 21(a)). Adopting $\rho = 0.1$ required 50 coarsening levels to yield $|V^1| = 52$ and $|V^1| = 53$ (Fig. 21(b)). Lower values of ρ imply in more coarsening levels to reach a target network size and additional memory to keep all the intermediate levels. It is also evident that the choice of ρ affects super-vertex balance, e.g., $\rho = 0.5$ yielded more balanced super-vertices (Fig. 21(a)) than $\rho = 0.1$ (Fig. 21(b)). A visual analysis reveals that the original global topological structures have been lost, an additional evidence that beyond favoring unbalanced super-vertices pairwise matching tends to degrade relevant topological structures. On the contrary, the models coarsened with *MLPb*, albeit losing detail, can actually reveal topological patterns that remain hidden, or blurred, in drawings of the full network.

We also investigated *MLPb* coarsening of non-symmetric sparse matrices with hole topology. A network hole is defined

as a loop formed by a vertex sub-set with only cyclic edges and no edges crossing the loop. Holes can indicate a special large-scale event occurring at a particular region, which may need monitoring. On communication networks, for instance, operation may be disrupted by sufficiently large holes [44]. Petri et al. [45] introduced a method to assist topological filtering methods by detecting particular structures, including network holes. Their method removes redundant edges to create reduced networks that fully preserve the topological information.

We illustrate the network labeled as “HB/jagmesh7” from the SuiteSparse Matrix Collection,¹³ shown in Fig. 20(a). Figs. 20(b) and 20(c) depict the network at two levels of coarsening, obtained setting $\mu = 0.1$ in algorithm *MLPb_{CSV}*, since we want balanced super-vertices. Notice the hole topology remains evident in the drawings of the coarser models. Our multilevel method can thus be useful, e.g., to verify the presence of holes inspecting coarser versions of a large-scale network, prior to performing complex or expensive tasks on views of the full model.

¹³ Available in <https://sparse.tamu.edu/>.

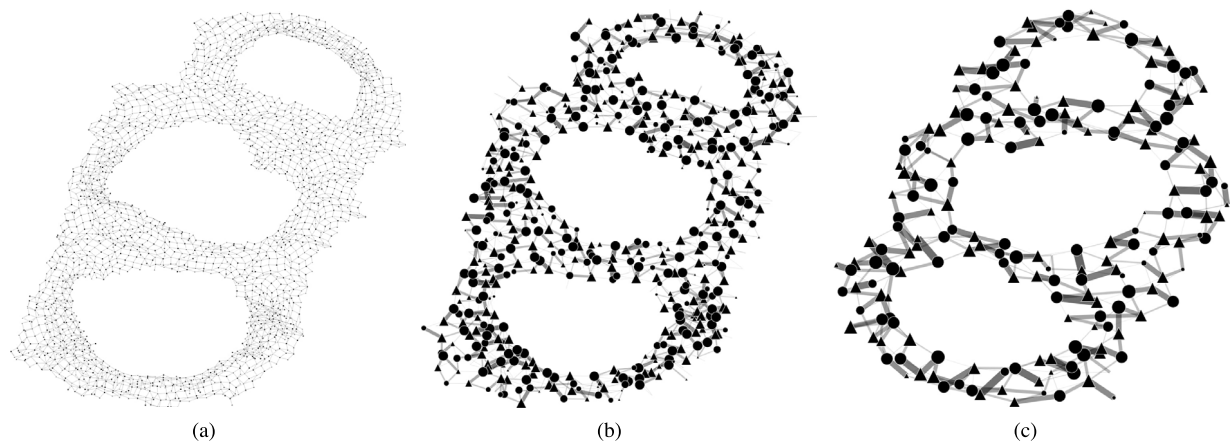


Fig. 20. Bipartite network with three holes: (a) shows the original network with $V^1 = V^2 = 1138$, (b) and (c) illustrate it at two increasing levels of coarsening, with $V^1 = V^2 = 300$ and $V^1 = V^2 = 100$, respectively.

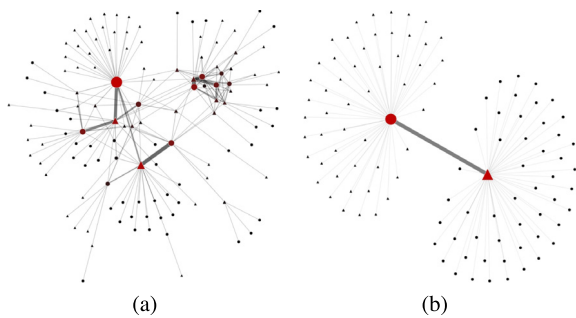


Fig. 21. Drawings of the N-reatome biological network coarsened with two settings of the RGMb matching algorithm. (a) illustrates the network after 11 coarsening levels, with $\rho = 0.5$, $|V^1| = 68$ and $|V^2| = 56$; (b) illustrates the network after 30 coarsening levels, with $\rho = 0.1$, $|V^1| = 52$ and $|V^2| = 53$.

Interestingly, the coarsened networks show holes that are similar in size to those of the original network. Although holes are defined with fewer edges in a coarser network, super-edge weights derive from the weights of their predecessor-edges, therefore the hole sizes are preserved in the force-directed layout. This example reinforces that $MLPb_{CSV}$ coarsening is robust to varied topological properties, as already exemplified in the scale-free, regular degree distribution and modular topological structures. Furthermore, its usage is more intuitive, as the user can specify a desired network reduction in terms of the number of vertices in each layer of the coarsest network.

According to Ghoniem et al. [34], even in executing simple tasks such as locating a vertex or finding the edges between two vertices, human perceptual abilities are limited to small network sizes. As networks increase, users find it difficult to accomplish even simple data exploration tasks. Thus, early interaction with simplified representations of a network can prove more useful to support early assessments than drawing the full network and all its associated information.

7. Concluding remarks

Inspired by the potential of general-purpose multilevel methods to scale optimization problems in networks we introduced a new coarsening strategy via weight-constrained label propagation for bipartite networks. We propose two variations that differ in the input parameters required: $MLPb_{CSL}$ that implements a standard formulation of the multilevel method, in which the user directly sets the number of desired levels and the network

reduction factor; and $MLPb_{CSV}$, which automatically creates the coarsening hierarchy given the desired number of vertices in the coarsest network.

We conducted three empirical studies for purposes of validating the $MLPb$ coarsening strategy. The first study describes its application to optimize the high-cost community detection algorithm $biSBM$. The results show that $MLPb_{CSL}$ can drastically speedup this classic community detection algorithm, while preserving solution quality. The second study applied $MLPb_{CSV}$ directly as a community detection algorithm, comparing the results thus obtained with those from three popular community detection algorithms known to be fast. Again, the results indicate that the $MLPb_{CSV}$ coarsening algorithm can yield equivalent solutions in terms of quality, at a lower computational cost.

Finally, in a third study of a more exploratory nature $MLPb_{CSV}$ was employed in connection with force-directed algorithms to display simplified drawings of several large or very large networks. The primary goal of this study was to illustrate the coarsening behavior of $MLPb_{CSV}$ in networks with distinct characteristics and topological properties. As a side effect, we argue that coarsening can be a useful resource in the context of interactive data visualization to support exploratory tasks on datasets represented as bipartite networks. There is evidence that the networks coarsened with $MLPb_{CSV}$ preserve the global topological patterns of the original models and allow a force-based drawing algorithm to build readable summarized layouts of the original. This strategy could be embedded into a visualization system, in connection with appropriate interaction functions, to enhance the user experience in scenarios that require interacting with node-link representations of large networks.

Identifying the level of coarsening which yields a suitable trade-off between algorithmic accuracy and execution times is a critical issue in applying the multilevel method. Currently, this is determined with *ad-hoc* investigations conducted in each application problem and dataset, but it is certainly an issue that deserves further investigation. $MLPb$ is the first coarsening solution to admit setting an expected size for the coarsest network, which is more intuitive than informing a number of coarsening levels. It is still necessary, however, to find out whether the coarsening applied is compatible with obtaining a good quality solution.

Our algorithms support a general implementation of the multilevel method in bipartite networks, and as such can encourage novel approaches to solve other problems beyond those addressed in this paper. We are particularly interested in its usage on supervised and unsupervised classification tasks, as well as in interactive visualization of large networks. Also as future work

we plan to extend the algorithms to handle problems defined in heterogeneous networks, where edges connect vertices of multiple types. The method's applicability could be thus extended to problems described by document-word networks indicating multiple types of associations, e.g. document-word, word-word, and document-document; or networks characterizing relationships between words, documents and authors. Investigating distributed or parallel paradigms for implementing the method is also a challenging research topic. An implementation of the proposed algorithms can be downloaded from <https://github.com/alanvalejo/mlpb>.¹⁴

CRedit authorship contribution statement

Alan Valejo: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing - original draft, Writing - review & editing. **Thiago Faleiros:** Formal analysis, Investigation, Methodology, Software, Writing - original draft, Writing - review & editing. **Maria Cristina Ferreira de Oliveira:** Supervision, Conceptualization, Methodology, Writing - original draft, Writing - review & editing. **Alneu de Andrade Lopes:** Supervision, Conceptualization, Methodology, Writing - original draft, Writing - review & editing.

Acknowledgments

This research was funded by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES), Brazil - Finance Code 001; the State of São Paulo Research Foundation (FAPESP), Brazil grants 15/14228-9 and 17/05838-3; and the Brazilian National Council for Scientific and Technological Development (CNPq) grants 302645/2015-2 and 301847/2017-7.

References

- [1] T.d.P. Faleiros, R.G. Rossi, A.A. Lopes, Optimizing the class information divergence for transductive classification of texts using propagation in bipartite graphs, *Pattern Recognit. Lett.* 87 (2017) 127–138.
- [2] T. Hwang, H. Sicotte, Z. Tian, B. Wu, J.-P. Kocher, D.A. Wigle, V. Kumar, R. Kuang, Robust and efficient identification of biomarkers by classifying features on graphs, *Bioinformatics* 24 (18) (2008) 2023–2029.
- [3] J. Grujić, Movies recommendation networks as bipartite graphs, in: *Proceedings of the International Conference on Computational Science (ICCS)*, Springer Berlin Heidelberg, 2008, pp. 576–583.
- [4] M.E.J. Newman, Scientific collaboration networks: I. network construction and fundamental results, *Phys. Rev. E* 64 (1) (2001) 1–8.
- [5] M.E.J. Newman, Scientific collaboration networks. II. Shortest paths, weighted networks, and centrality, *Phys. Rev. E* 64 (1) (2001) 016132.
- [6] A. Valejo, V. Ferreira, M.C.F. Oliveira, A.A. Lopes, Community detection in bipartite network: A modified coarsening approach, in: *Proceedings of the Information Management and Big Data*, Springer International Publishing, 2018, pp. 123–136.
- [7] R.G. Rossi, A.A. Lopes, T. de Paulo Faleiros, S.O. Rezende, Inductive model generation for text classification using a bipartite heterogeneous network, *J. Comput. Sci. Tech.* 29 (3) (2014) 361–375.
- [8] A.S. Asratian, T.M.J. Denley, R. Haggkvist, *Bipartite Graphs and their Applications*, Cambridge University Press, New York, NY, USA, 1998.
- [9] A. Valejo, J. Valverde-Rebaza, B. Drury, A.A. Lopes, Multilevel refinement based on neighborhood similarity, in: *Proceedings of the International Database Engineering & Applications Symposium (IDEAS)*, in: *IDEAS '14*, ACM, New York, NY, USA, 2014, pp. 67–76.
- [10] A. Valejo, J. Valverde-Rebaza, A.A. Lopes, A multilevel approach for overlapping community detection, in: *Proceedings of the Brazilian Conference on Intelligent Systems (BRACIS)*, 2014, pp. 390–395.
- [11] A. Valejo, V. Ferreira, G.P.R. Filho, M.C.F. Oliveira, A.A. Lopes, One-mode projection-based multilevel approach for community detection in bipartite networks, in: *Proceedings of the International Symposium on Information Management and Big Data (SIMBig)*, 2017, pp. 101–108.
- [12] A. Valejo, M.C.F. Oliveira, G.P.R. Filho, A.A. Lopes, Multilevel approach for combinatorial optimization in bipartite network, *Knowl.-Based Syst.* 151 (2018) 45–61.
- [13] D.B. Larremore, A. Clauset, A.Z. Jacobs, Efficiently inferring community structure in bipartite networks, *Phys. Rev. E* 90 (1) (2014) 012805.
- [14] A.H. Gebremedhin, F. Manne, A. Pothen, Graph Coloring in Optimization Revisited, in: *Reports in Informatics*, Department of Informatics, University of Bergen, 2002.
- [15] A.H. Gebremedhin, F. Manne, A. Pothen, Parallel distance-k coloring algorithms for numerical optimization, in: *Proceedings of the 8th International Euro-Par Conference on Parallel Processing*, in: *Euro-Par '02*, 2002, pp. 912–921.
- [16] L. Lu, T. Zhou, Link prediction in weighted networks : The role of weak ties, *Europhys. Lett. (EPL)* 89 (1) (2010) 18001.
- [17] G. Karypis, V. Kumar, A fast and high quality multilevel scheme for partitioning irregular graphs, *SIAM J. Sci. Comput. (SISC)* 20 (1) (1998) 359–392.
- [18] K.A. Zweig, M. Kaufmann, A systematic approach to the one-mode projection of bipartite graphs, *Soc. Netw. Anal. Min.* 1 (3) (2011) 187–218.
- [19] S. Banerjee, M. Jenamani, D.K. Pratihari, Properties of a projected network of a bipartite network, in: *2017 International Conference on Communication and Signal Processing (ICCSP)*, IEEE, 2017, pp. 0143–0147.
- [20] U.N. Raghavan, R. Albert, S. Kumara, Near linear time algorithm to detect community structures in large-scale networks, *Phys. Rev. E* 76 (2007) 036106.
- [21] A. Valejo, F. Goes, L.M. Romanetto, M.C.F. Oliveira, A.A. Lopes, A benchmarking tool for the generation of bipartite network models with overlapping communities, *Knowl. Inf. Syst.* (2019) accepted paper.
- [22] Z. Xiaoqin, G. Zoubin, Learning from Labeled and Unlabeled Data with Label Propagation, Tech. Rep., Technical Report CMU-CALD-02-107, Carnegie Mellon University, 2002.
- [23] K. Zhoua, A. Martin, Q. Pan, Z. Liu, Evidential label propagation algorithm for graphs, in: *Proceedings of the International Conference on Information Fusion (FUSION)*, 2016, pp. 1316–1323.
- [24] V. Labatut, Generalized measures for the evaluation of community detection methods, *Int. J. Soc. Netw. Anal. Min. (SNAM)* 2 (1) (2015) 44–63.
- [25] J. Demšar, Statistical comparisons of classifiers over multiple data sets, *J. Mach. Learn. Res.* 7 (2006) 1–30.
- [26] M.J. Barber, J.W. Clark, Detecting network communities by propagating labels under constraints, *Phys. Rev. E* 80 (2009) 026129.
- [27] V.D. Blondel, J.-L. Guillaume, R. Lambiotte, E. Lefebvre, Fast unfolding of communities in large networks, *J. Stat. Mech. Theory Exp.* 10 (2008) 10008.
- [28] A. Clauset, M.E.J. Newman, C. Moore, Finding community structure in very large networks, *Phys. Rev. E* 70 (2004) 066111.
- [29] S. Fortunato, Community detection in graphs, *Phys. Rep.* 486 (3–5) (2010) 75–174.
- [30] A. Chessa, I. Crimaldi, M. Riccaboni, L. Trapin, Cluster analysis of weighted bipartite networks: A new copula-based approach, *PLOS ONE* 9 (10) (2014) 1–12.
- [31] T. Alzahrani, K.J. Horadam, Community detection in bipartite networks: Algorithms and case studies, in: J. Lu, X. Yu, G. Chen, W. Yu (Eds.), *Proceeding of the Complex Systems and Networks: Dynamics, Controls and Applications*, 2016, pp. 25–50.
- [32] T.J.B. Cann, I.S. Weaver, H.T.P. Williams, Is it correct to project and detect? assessing performance of community detection on unipartite projections of bipartite networks, in: *Complex Networks and their Applications VII*, Springer International Publishing, 2019, pp. 267–279.
- [33] T. Chakraborty, A. Dalmia, A. Mukherjee, N. Ganguly, Metrics for community analysis: A survey, *ACM Comput. Surv.* 50 (4) (2017) 54:1–54:37.
- [34] M. Ghoniem, J.-D. Fekete, P. Castagliola, On the readability of graphs using node-link and matrix-based representations: A controlled experiment and statistical analysis, *Inf. Vis.* 4 (2) (2005) 114–135.
- [35] J. Díaz, J. Petit, M. Serna, A survey of graph layout problems, *ACM Comput. Surv.* 34 (3) (2000) 313–356.
- [36] S. Hachul, M. Jünger, Large-graph layout algorithms at work: An experimental study, *J. Graph Algorithms Appl.* 11 (2007) 345–369.
- [37] T. von Landesberger, A. Kuijper, T. Schreck, J. Kohlhammer, J.J. van Wijk, J.-D. Fekete, D.W. Fellner, Visual analysis of large graphs: State-of-the-art and future research challenges, *Comput. Graph. Forum* 30 (6) (2011) 1719–1749.
- [38] F.G. Toosi, N.S. Nikolov, Vertex-neighboring multilevel force-directed graph drawing, in: *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2016, pp. 002996–003001.
- [39] M.D. Dias, M.R. Mansour, F. Dias, F. Petronetto, C.T. Silva, L.G. Nonato, A hierarchical network simplification via non-negative matrix factorization, in: *Proceedings of the Conference on Graphics, Patterns and Images (SIBGRAPI)*, 2017, pp. 119–126.
- [40] A. Perrot, D. Auber, Cornac: Tackling huge graph visualization with big data infrastructure, *IEEE Trans. Big Data* 14 (2018) 1.

¹⁴ The software will be made available immediately after paper acceptance.

- [41] P. Eades, A heuristic for graph drawing, *Congr. Numer.* 42 (1984) 149–160.
- [42] T. Kamada, S. Kawai, An algorithm for drawing general undirected graphs, *Inf. Process. Lett.* 31 (1) (1989) 7–15.
- [43] T.M.J. Fruchterman, E.M. Reingold, Graph drawing by force-directed placement, *Softw. Pract. Exper.* 21 (11) (1991) 1129–1164.
- [44] S. Funke, Topological hole detection in wireless sensor networks and its applications, in: *Proceedings of the 2005 Joint Workshop on Foundations of Mobile Computing*, in: DIALM-POMC '05, 2005, pp. 44–53.
- [45] G. Petri, M. Scolamiero, I. Donato, F. Vaccarino, Topological strata of weighted complex networks, *PLOS ONE* 8 (6) (2013) 1–8.